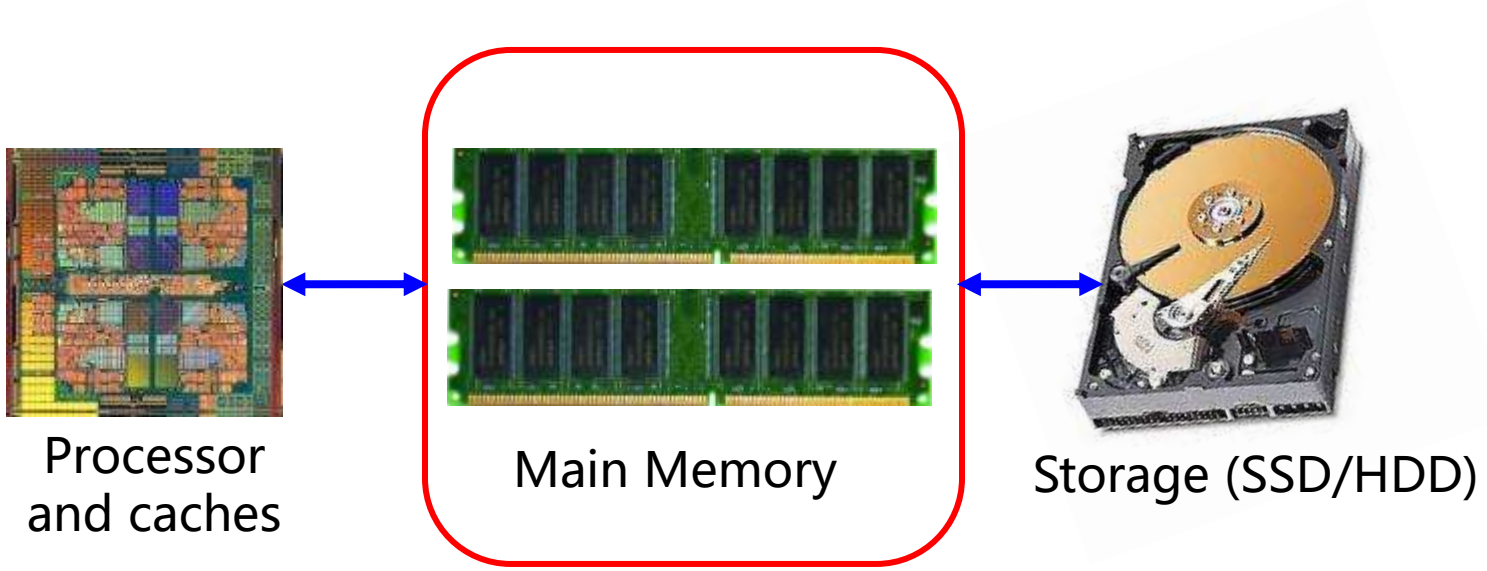
计算机体系结构

06. 虚拟存储器

李建华
计算机与信息学院

The contents of the slides are adapted from CA course of wisc, princeton, mit, berkeley, edinburg, and eth. The uses of the slides of this course are for educational purposes only and should be used only in conjunction with the textbook. Derivatives of the slides must acknowledge the copyright notices of this and the originals.

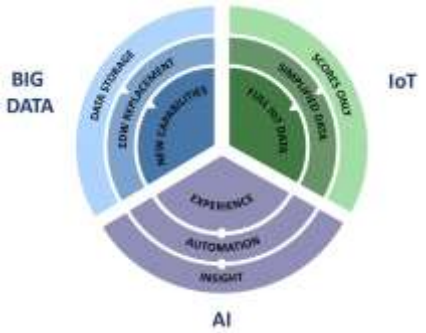
主存 (Main Memory)



- 主存是所有计算系统的核心部件之一
  - Server
  - Mobile
  - Embedded
  - Desktop
  - Sensor

主存的现状

- 当前的技术、架构和应用趋势
  - 对主存产生了一些**新的需求**
  - **放大了**主存以往存在的旧问题



- 导致基于DRAM的内存难以满足应用的需求
- 新型的非易失性存储器 (如PCM) 带来新的机遇
  - 内存和存储的融合 -> 带来新的变化
- 当前，内存系统的主要研究方向：
  - 解决或优化DRAM面临的问题，满足应用需求
  - 使得新型技术尽快可用，替换掉当前的DRAM技术

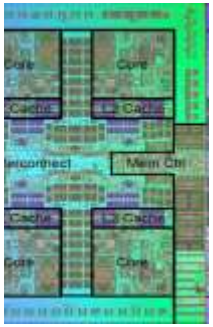
对主存容量的需求

- 更多的核心 ➔ 更多的并发性 ➔ 更大的工作集

Why NVIDIA H100?

The **NVIDIA H100 GPU** includes the following units:

- 7 or 8 GPCs, 57 TPCs, 2 SMs/TPC, 114 SMs per GPU
- 128 FP32 CUDA Cores/SM, 14592 FP32 CUDA Cores per GPU
- 4 4th-generation Tensor Cores per SM, 456 per GPU
- 80 GB HBM2e, 5 HBM2e stacks, 10 512-bit memory controllers
- 50 MB L2 cache
- Fourth-generation NVLink and PCIe Gen 5 Support



: 8 cores

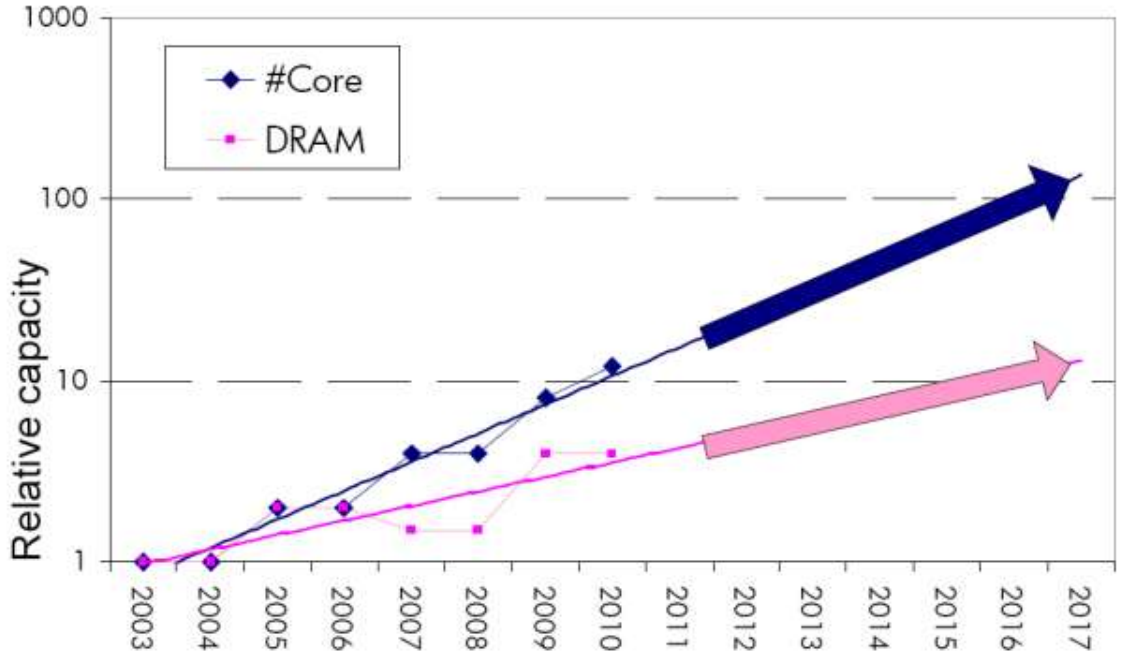


AMD EPYC 9965: 192 cores

- 越来越多的应用是**数据密集型**的 -> 更大的内存容量
- 很多 应用/虚拟机 会**共享**内存：
  - 数据中心Servers: 融合不同应用，**共享**内存；
  - GP-GPUs: 数量庞大的并行线程，**共享**内存；
  - Mobile: 交互性和非交互性应用融合，**共享**内存。

内存容量和应用需求之间的差距

- CPU中集成的核的**数量**约每2年翻番
- DRAM DIMM 的**容量**约每3年翻番 – 越来越难



- 每个核所拥有的内存大小每2年减少30%
- 每个核所拥有的内存带宽情况更加严峻!

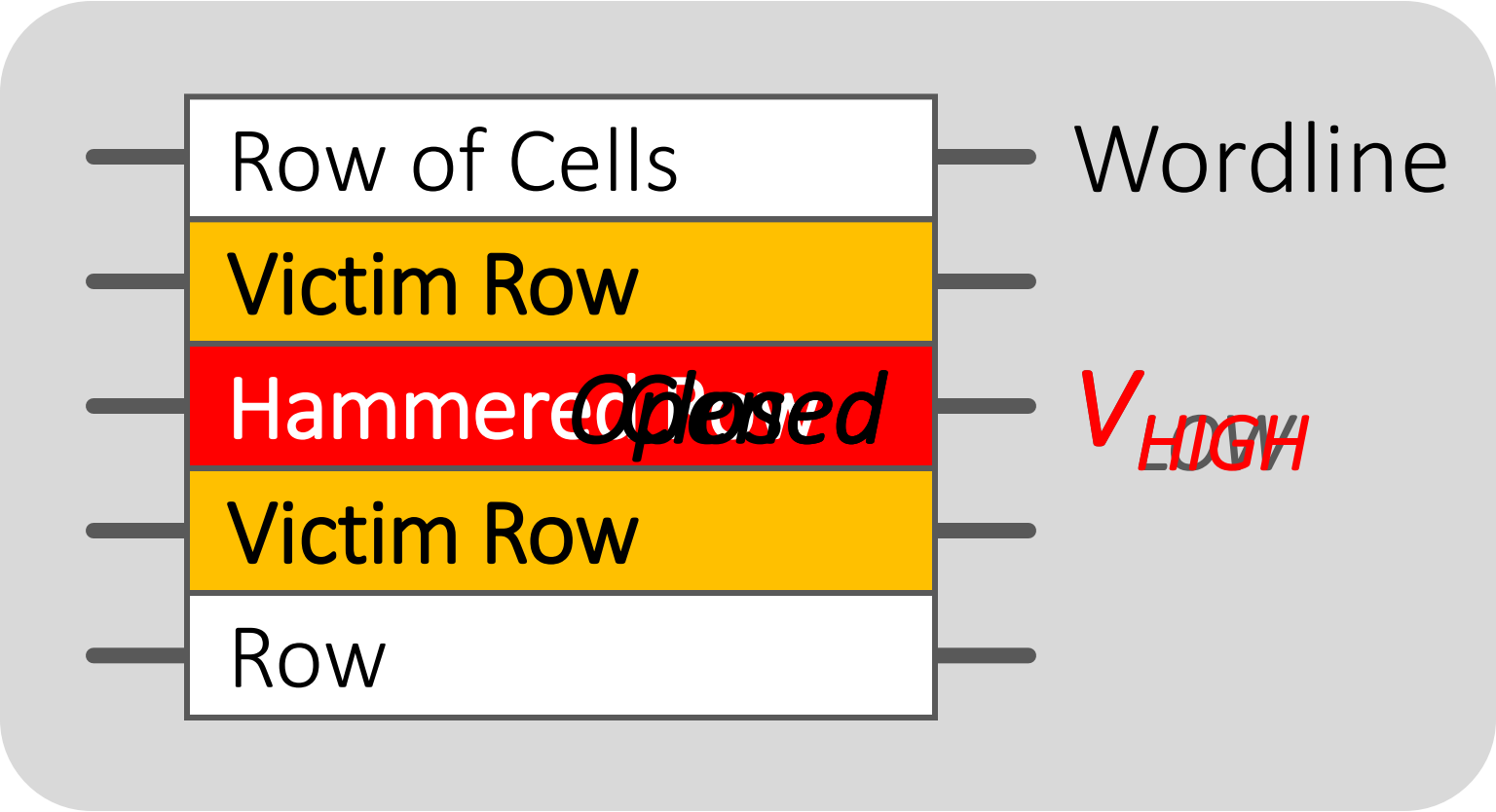
DRAM内存面临的问题：能耗问题

- 内存的**能耗**是另一个逐渐重要的核心问题
  - IBM服务器: ~50% 功耗用于内存系统 [Lefurgy, IEEE Computer 2003]
  - DRAM 在空闲状态下也耗能 – leakage power
  - 此外，DRAM周期性的刷新操作也耗能；

- DRAM技术的扩展之路很快将到达终点

C. Lefurgy, et al., “Energy management for commercial servers”, in IEEE Computer, Volume: 36, Issue: 12, 2003.

DRAM的可靠性问题：Disturbance Errors

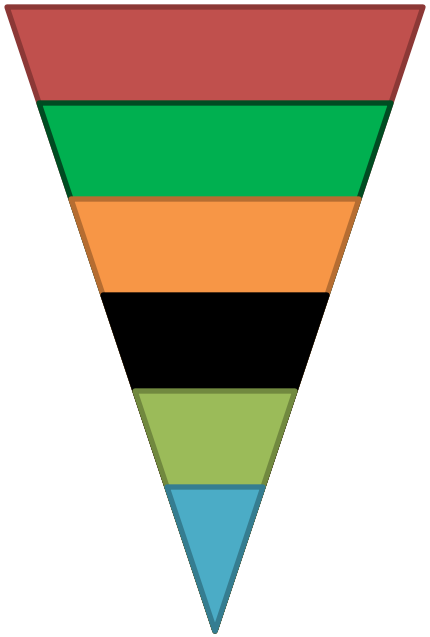


Repeatedly reading a row enough times (before memory gets refreshed) induces disturbance errors in **adjacent rows** in most real DRAM chips you can buy today

Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors, (Kim et al., ISCA 2014)

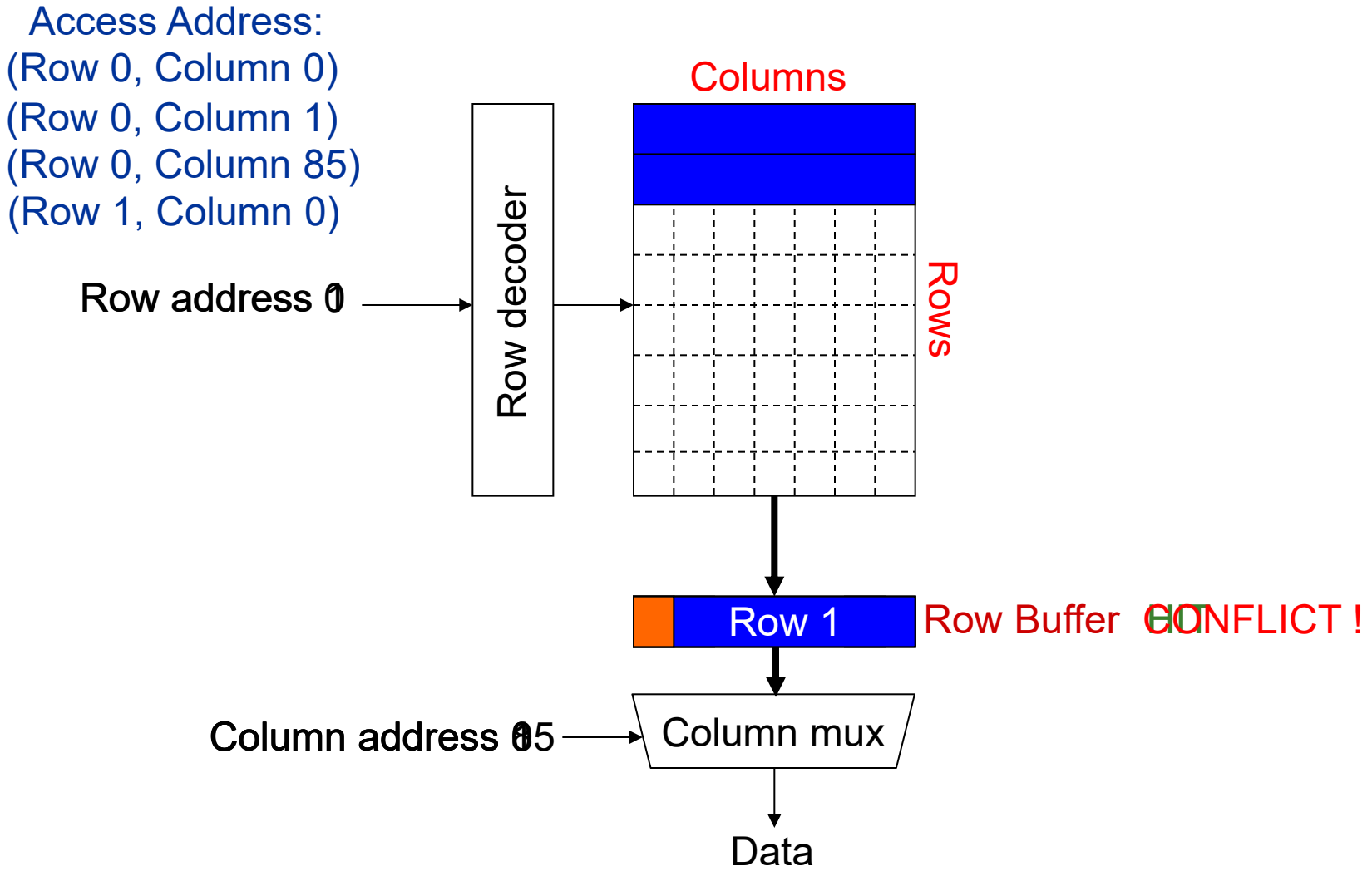
DRAM系统的组织

- Channel
- DIMM
- Rank
- Chip
- Bank
- Row/Column
- Cell



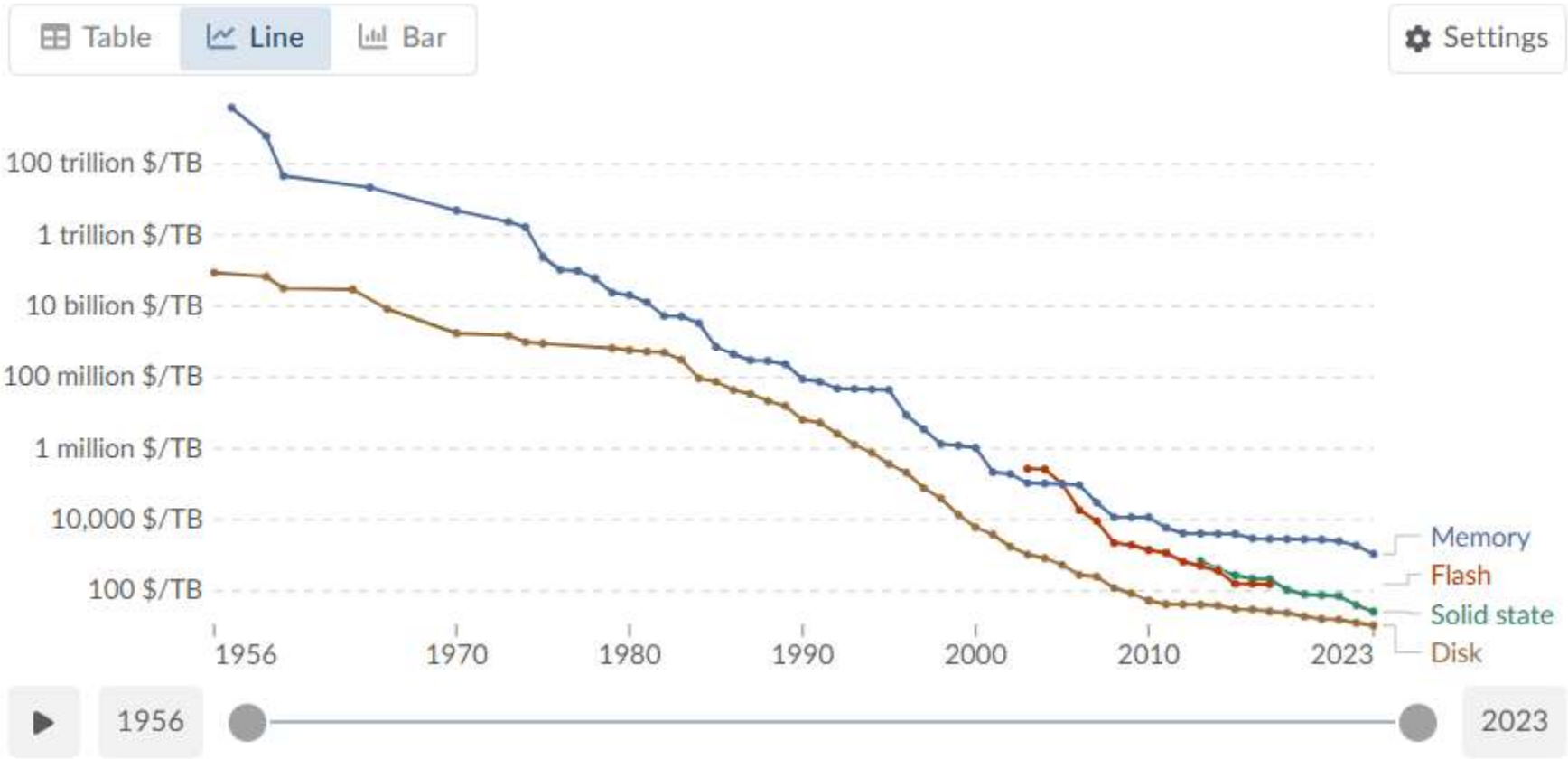
探索主内存，以DDR5为例：
https://www.bilibili.com/video/BV1vP411c7pt/?spm_id_from=..search-card.all.click&vd_source=f4b86ceb9b5012adaf35e95eb51f5e7b

Bank的组织


Access Address:
(Row 0, Column 0)
(Row 0, Column 1)
(Row 0, Column 85)
(Row 1, Column 0)
Columns
Row address 1
Row decoder
Rows
Row 1
Row Buffer
CONFLICT !
Column address 85
Column mux
Data

Chip的组织

- 由多个bank构成 (通常是8个)
- Chip内的bank共享三个总线

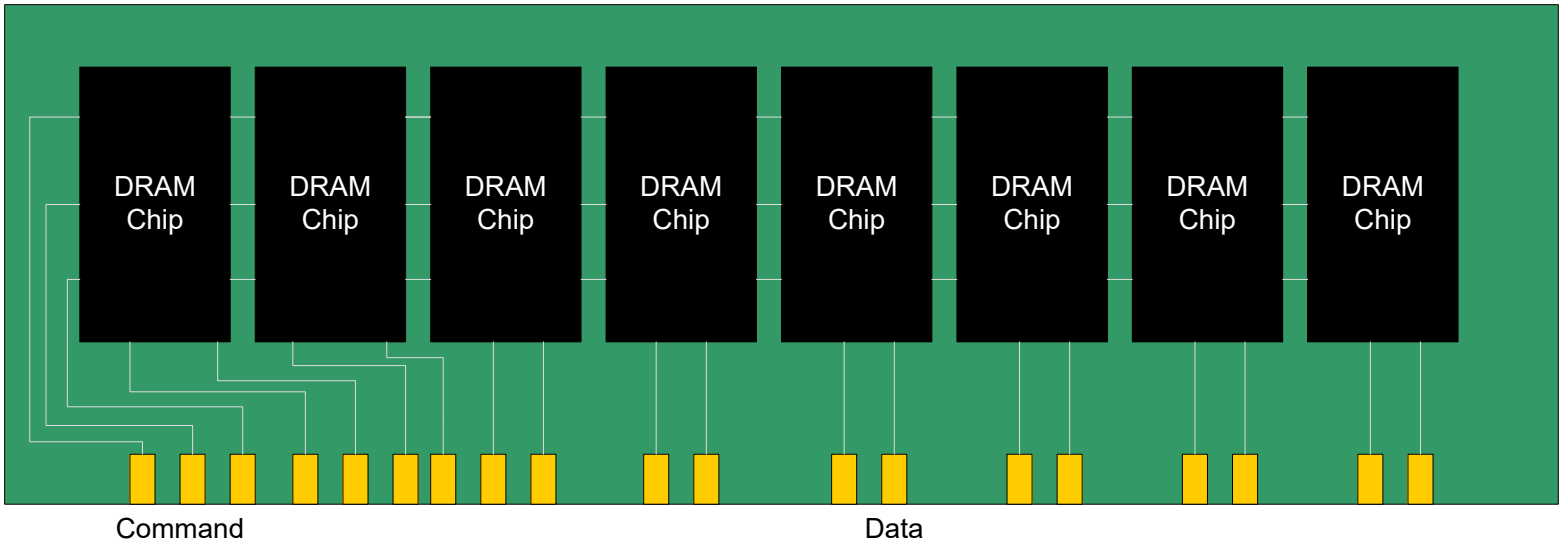


https://ourworldindata.org/grapher/historical-cost-of-computer-memory-and-storage?time=earliest..2023

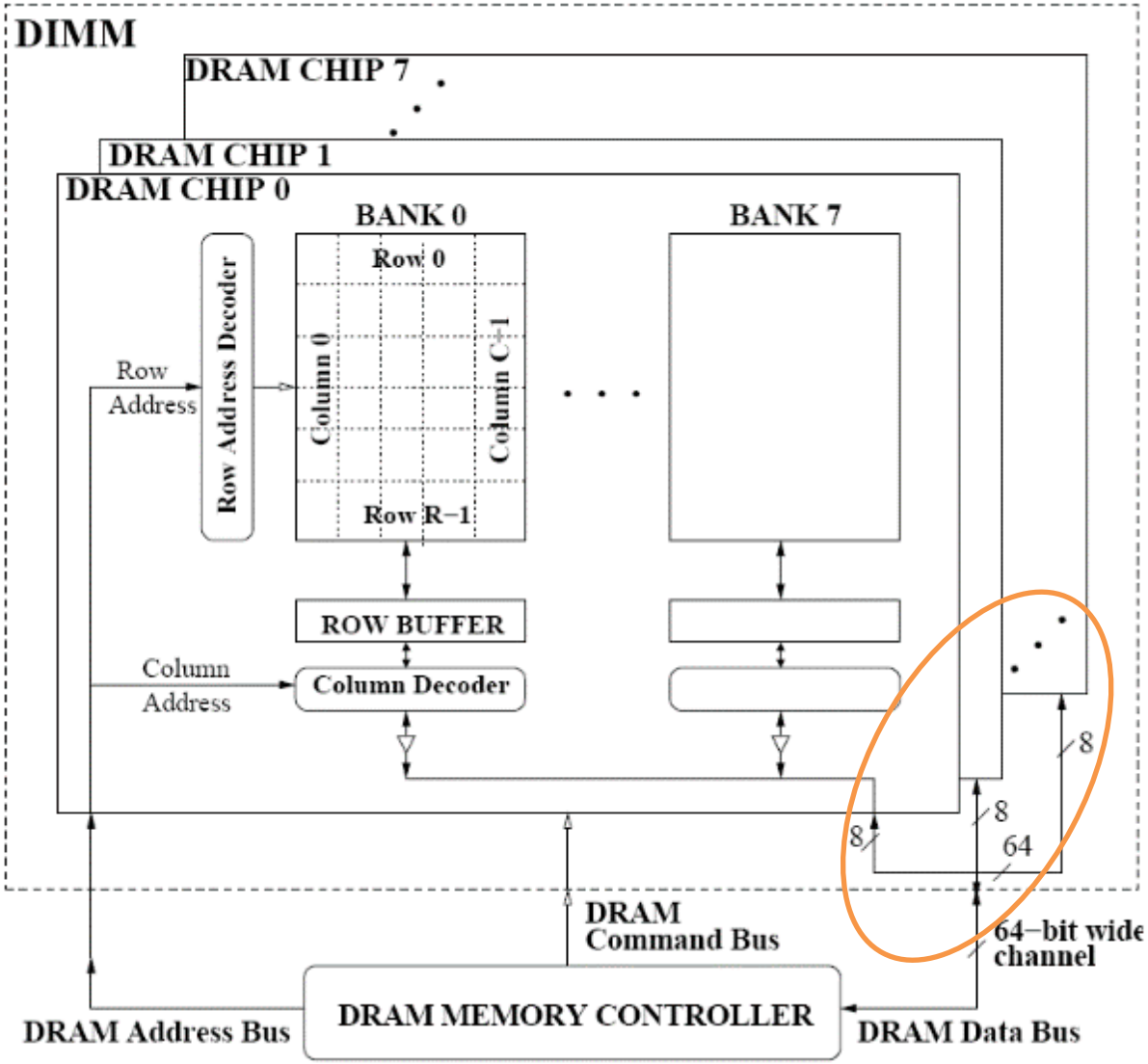
Rank 和 Module

- Rank: 多个chip在一起协同操作形成一个更宽的接口（通常是8个Chip）
- 构成一个rank的所有chip由相同的信号控制
  - 响应同一个命令
  - **共享**地址和命令总线
  - **不共享**数据总线
- 一个DRAM Module由一个或多个rank构成
  - 例如，DIMM (dual inline memory module)
  - 就是当前插到主板上的内存条

A 64-bit Wide DIMM (One Rank)

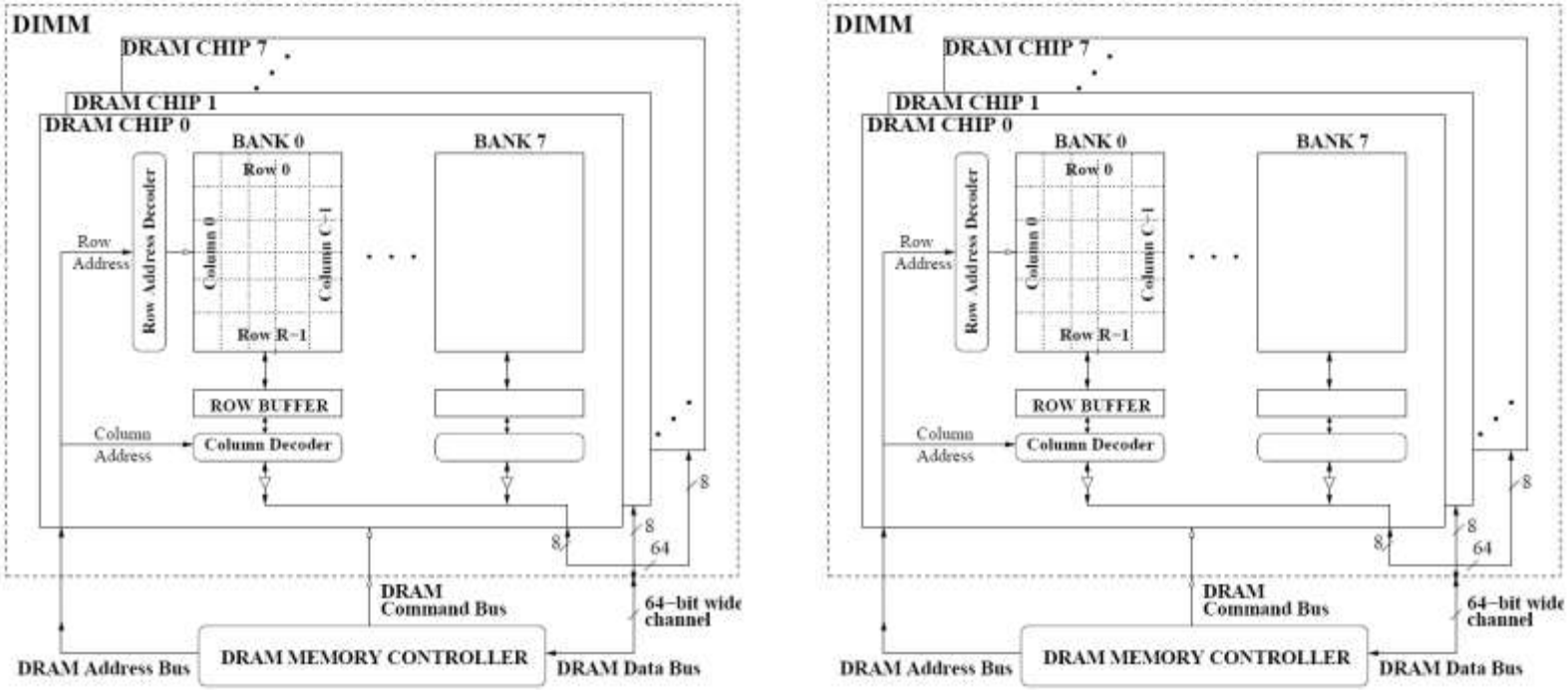


A 64-bit Wide DIMM (One Rank)



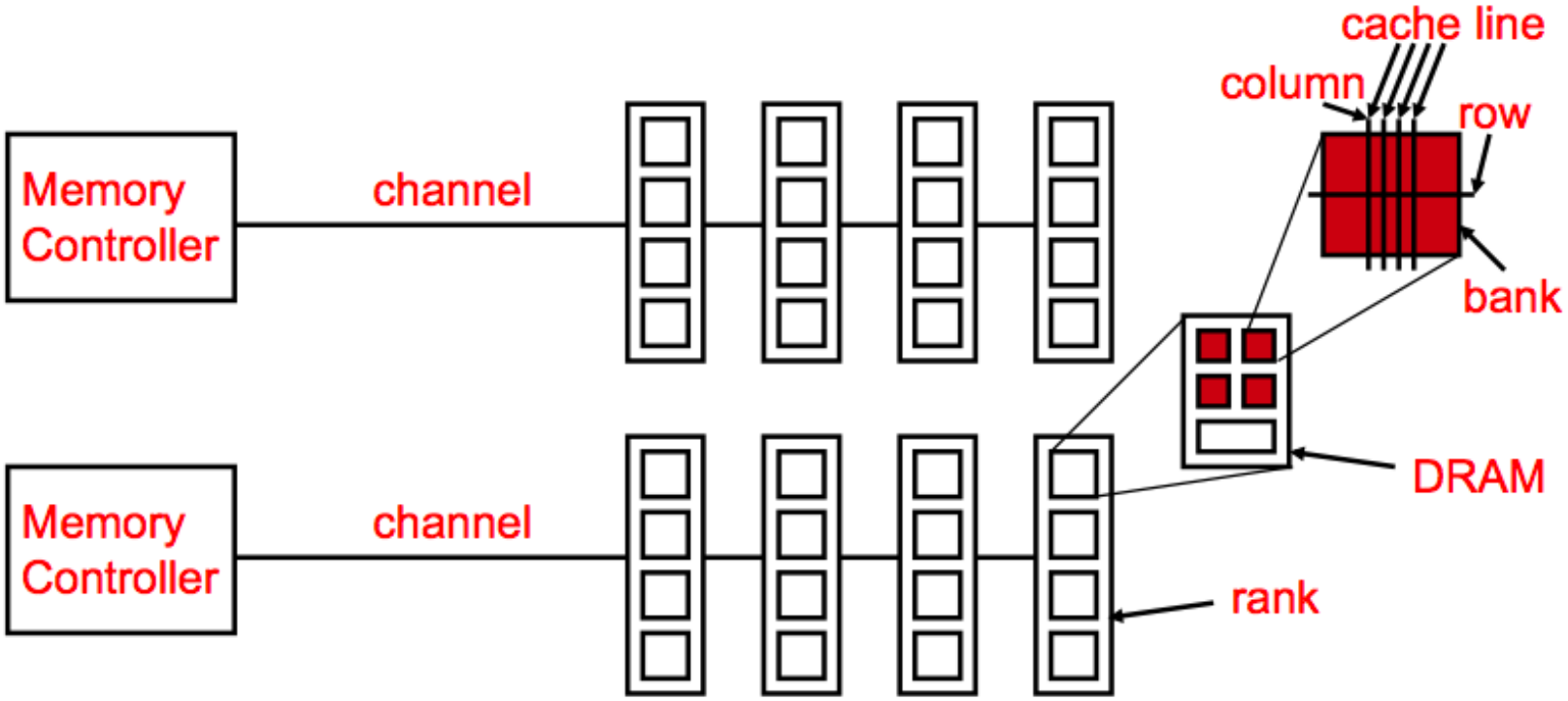
- 优点:
  - 操作起来像一个宽接口的大容量DRAM chip
  - 内存控制器不需要处理单个chip
- 缺点:
  - 访问的粒度固定，不能小于接口的宽度，灵活性有限。

Channels

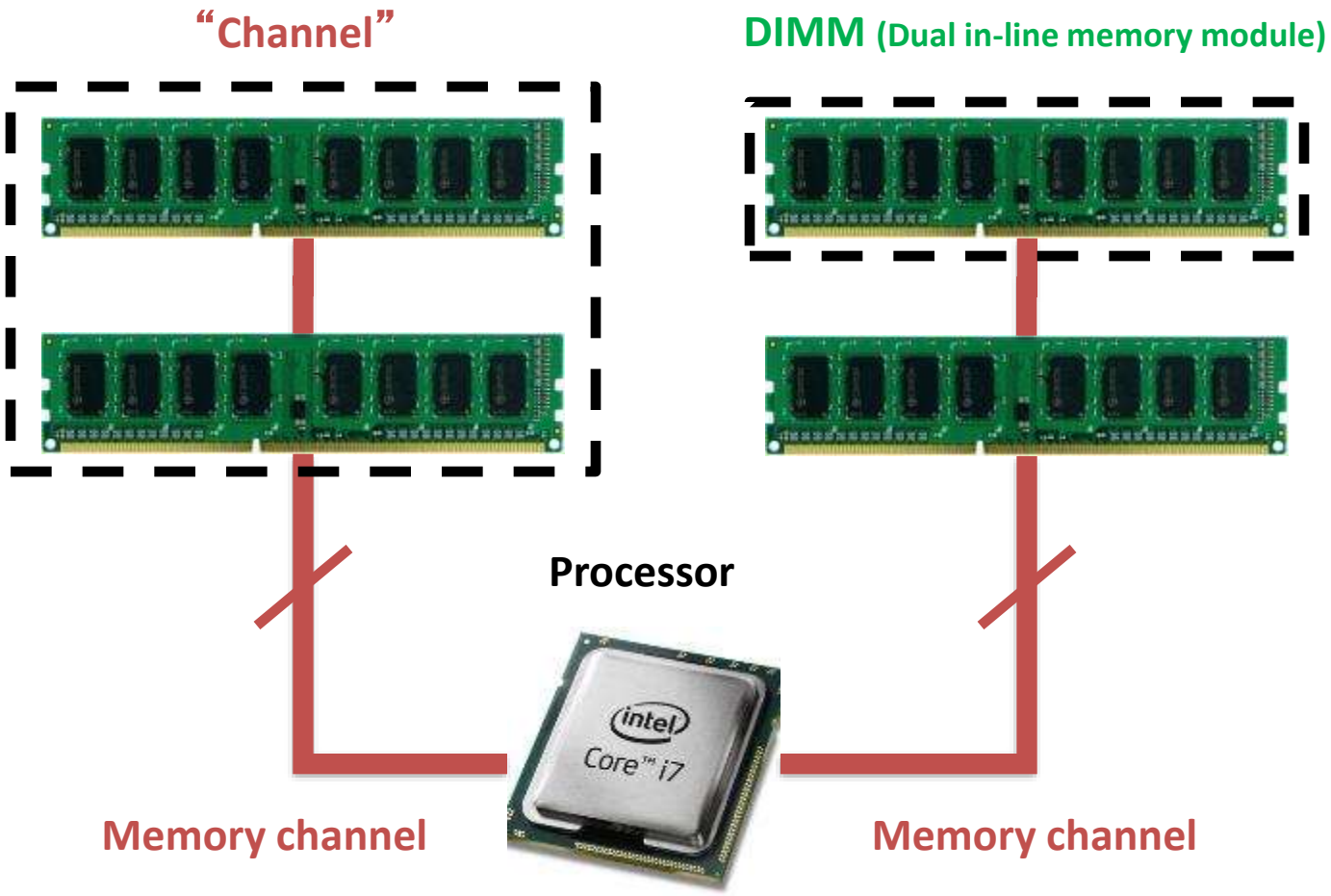


- 2个独立channel: 2个内存控制器

内存架构图


Memory Controller
channel
Memory Controller
channel
cache line
column
row
bank
DRAM
rank

DRAM系统

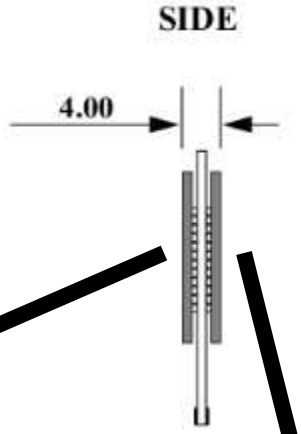

“Channel”
DIMM (Dual in-line memory module)
Processor
intel
Core™ i7
Memory channel
Memory channel

DIMM的拆分

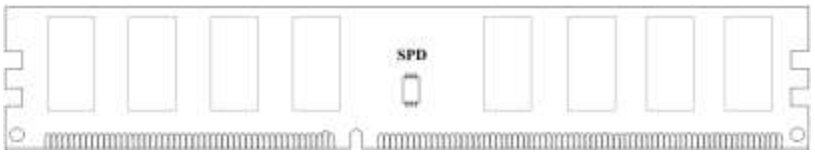
DIMM (Dual in-line memory module)



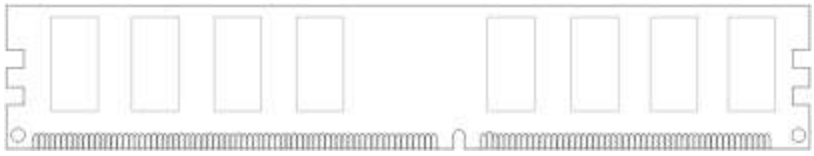
Side view



Front of DIMM



Back of DIMM

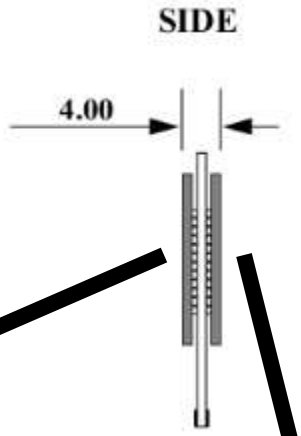


DIMM的拆分

DIMM (Dual in-line memory module)

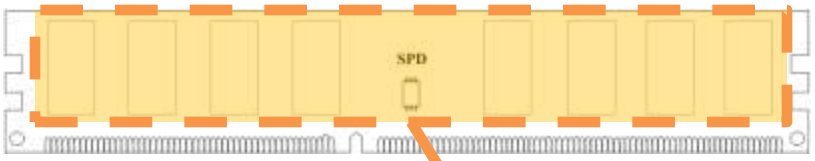


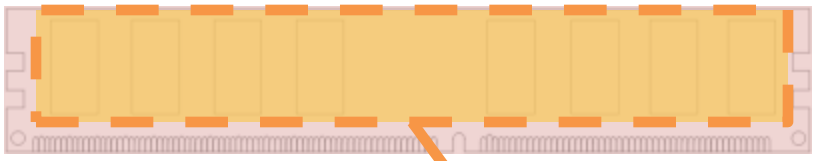
Side view



Front of DIMM

Back of DIMM

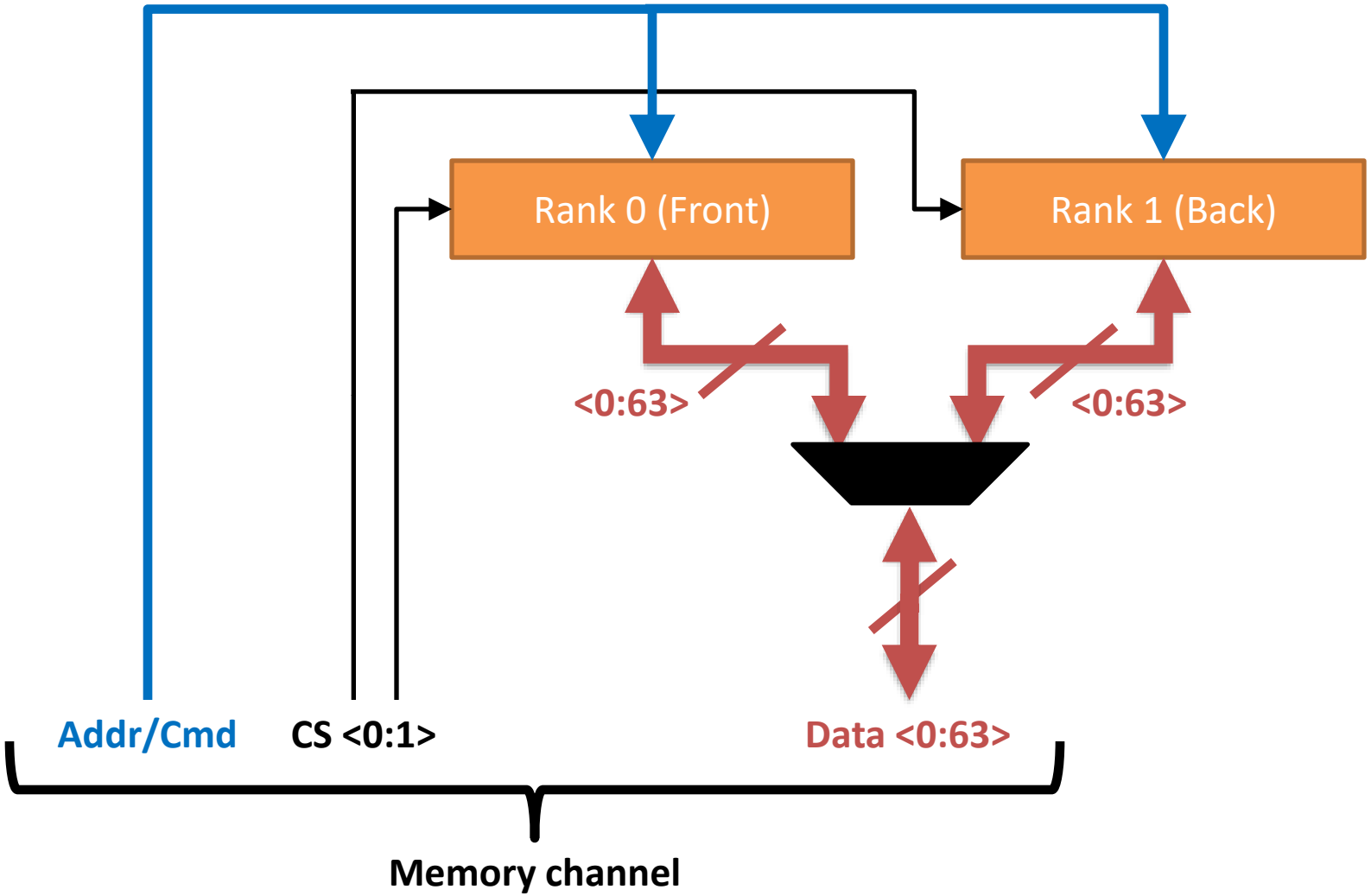




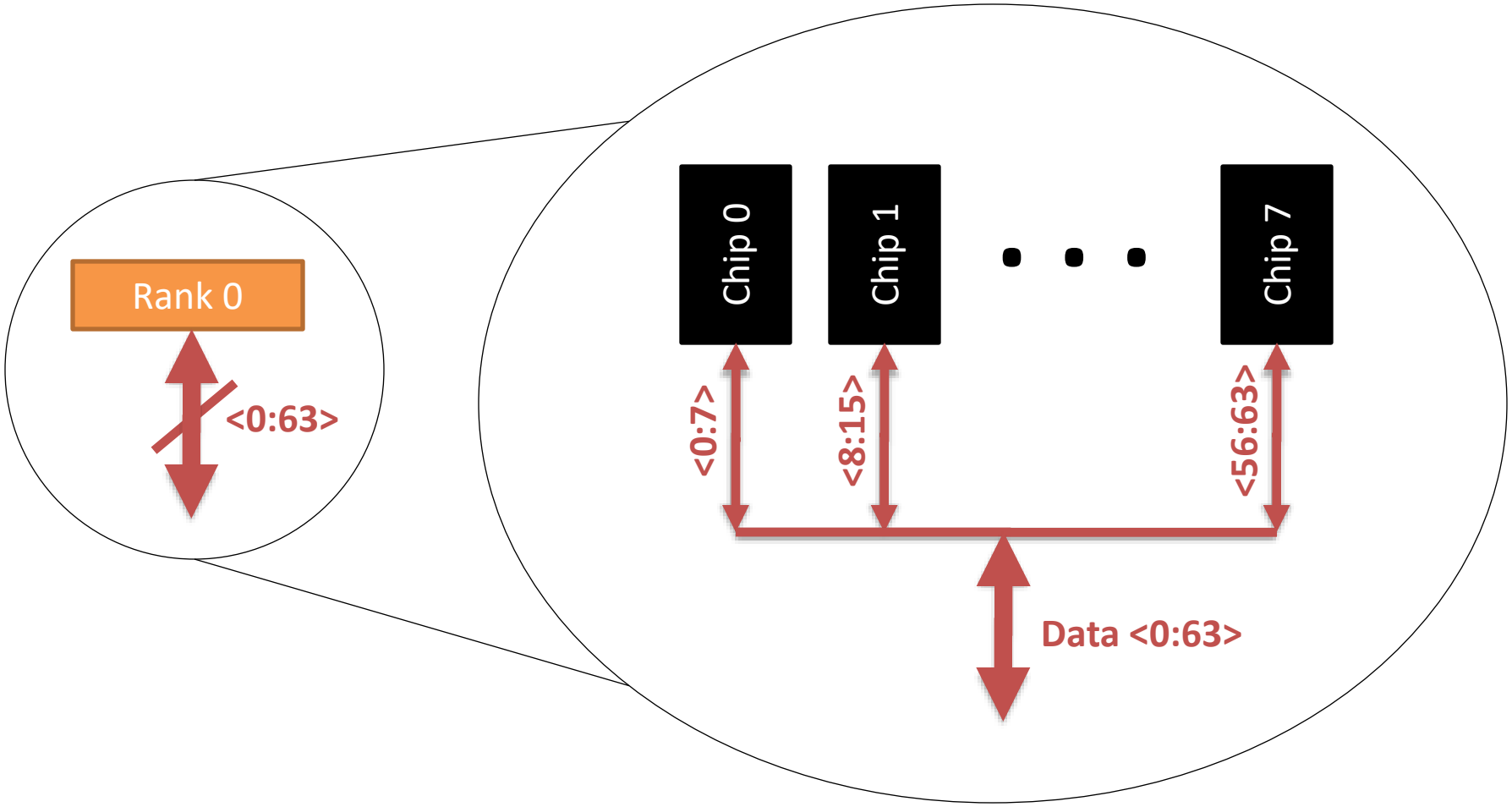
Rank 0: collection of 8 chips

Rank 1

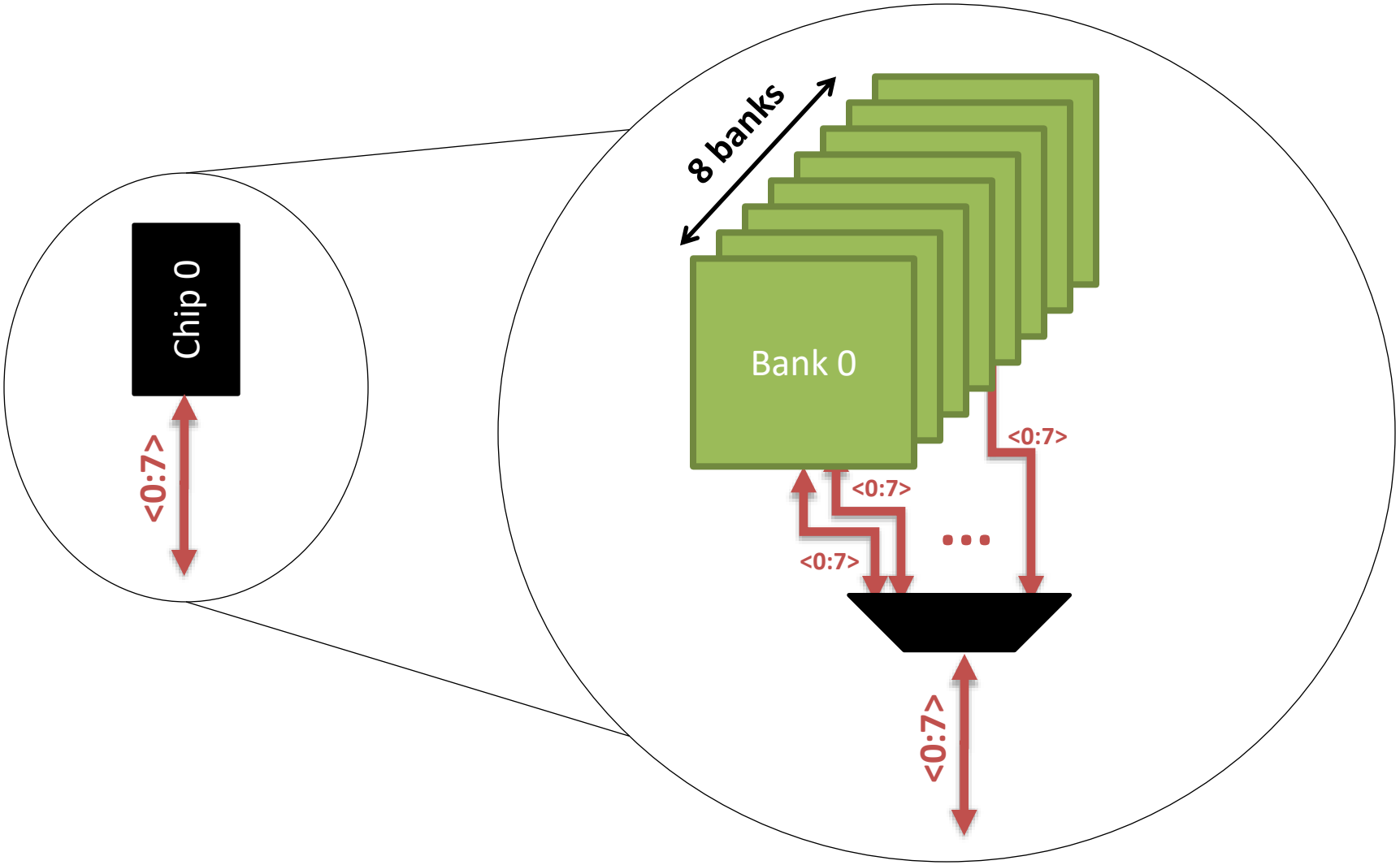
RANK的逻辑架构


Rank 0 (Front)
Rank 1 (Back)
<0:63>
<0:63>
Addr/Cmd
CS <0:1>
Data <0:63>
Memory channel

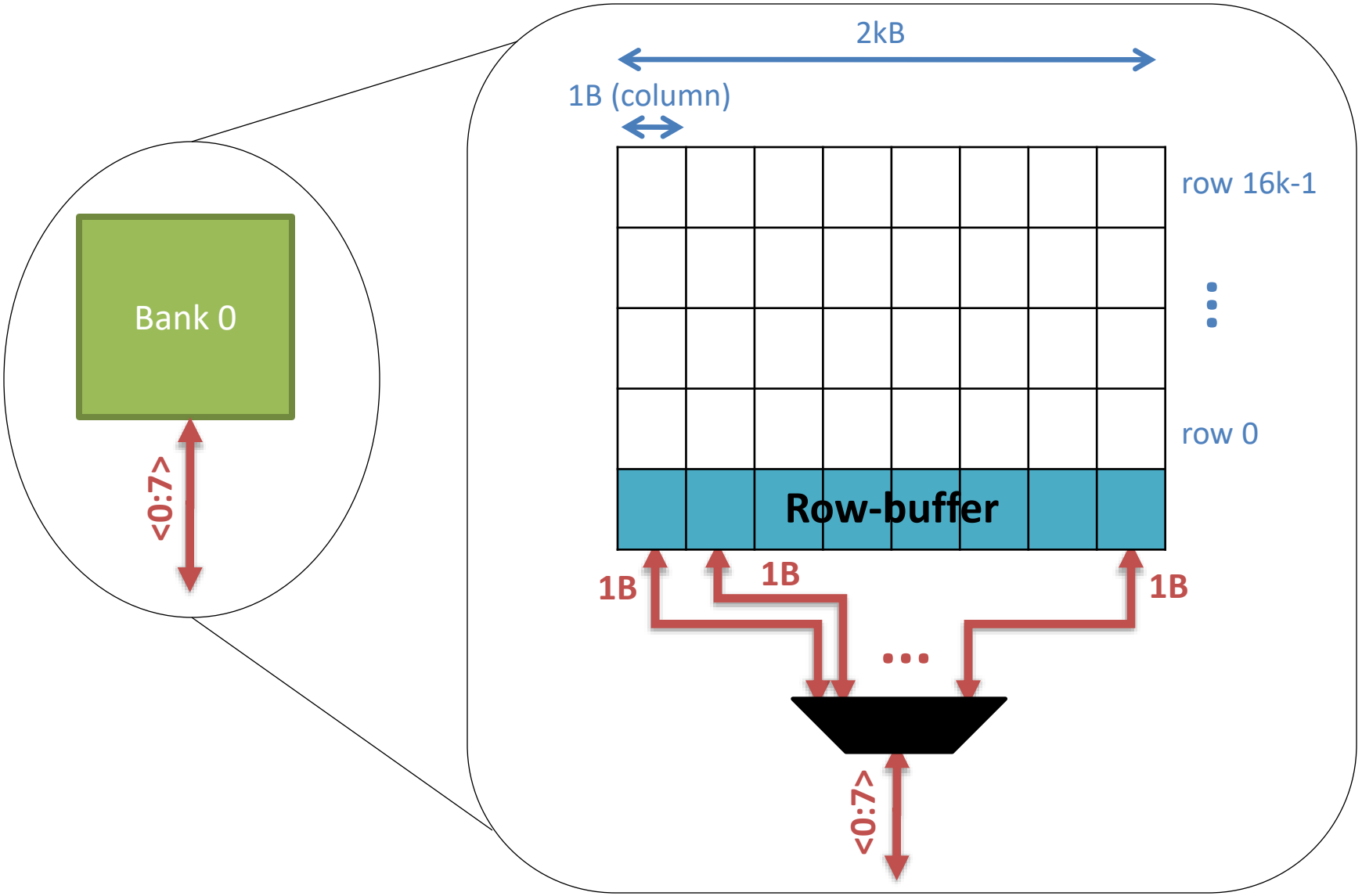
RANK的拆分



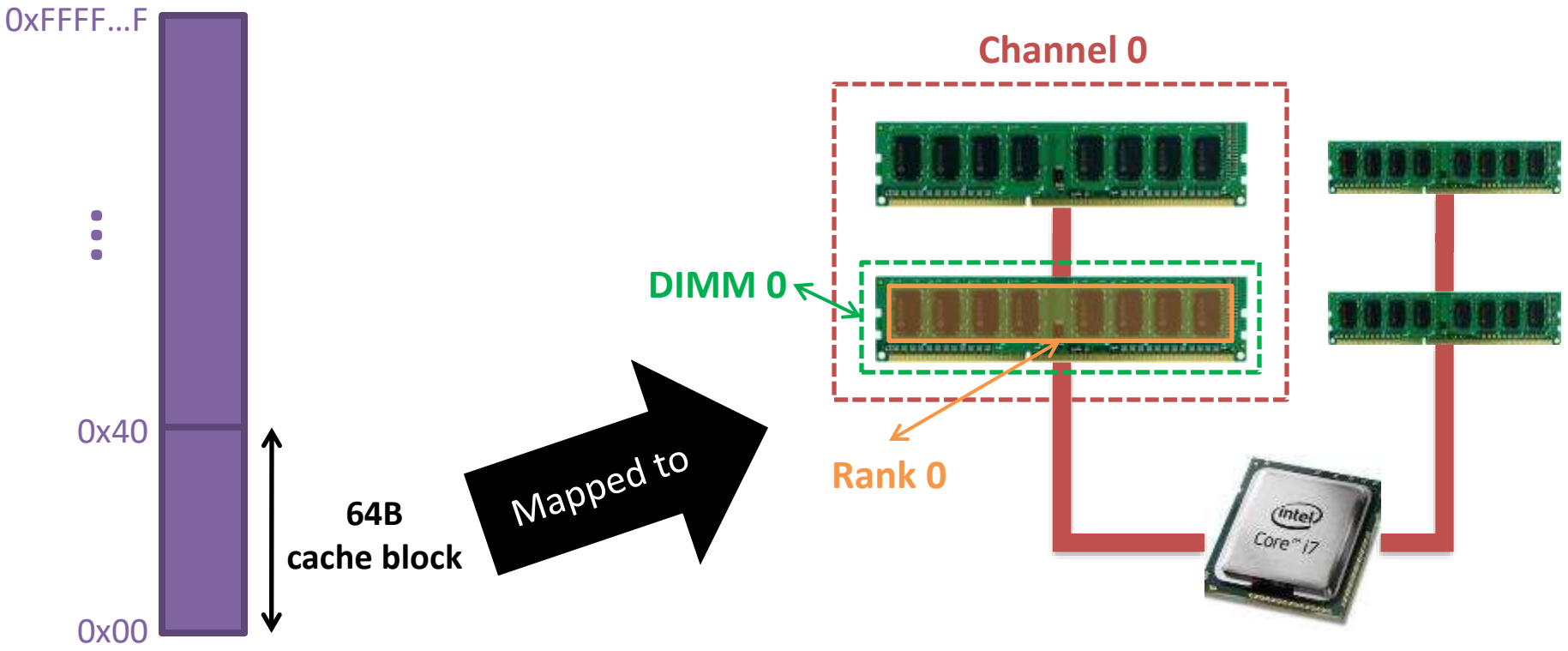
Chip的拆分



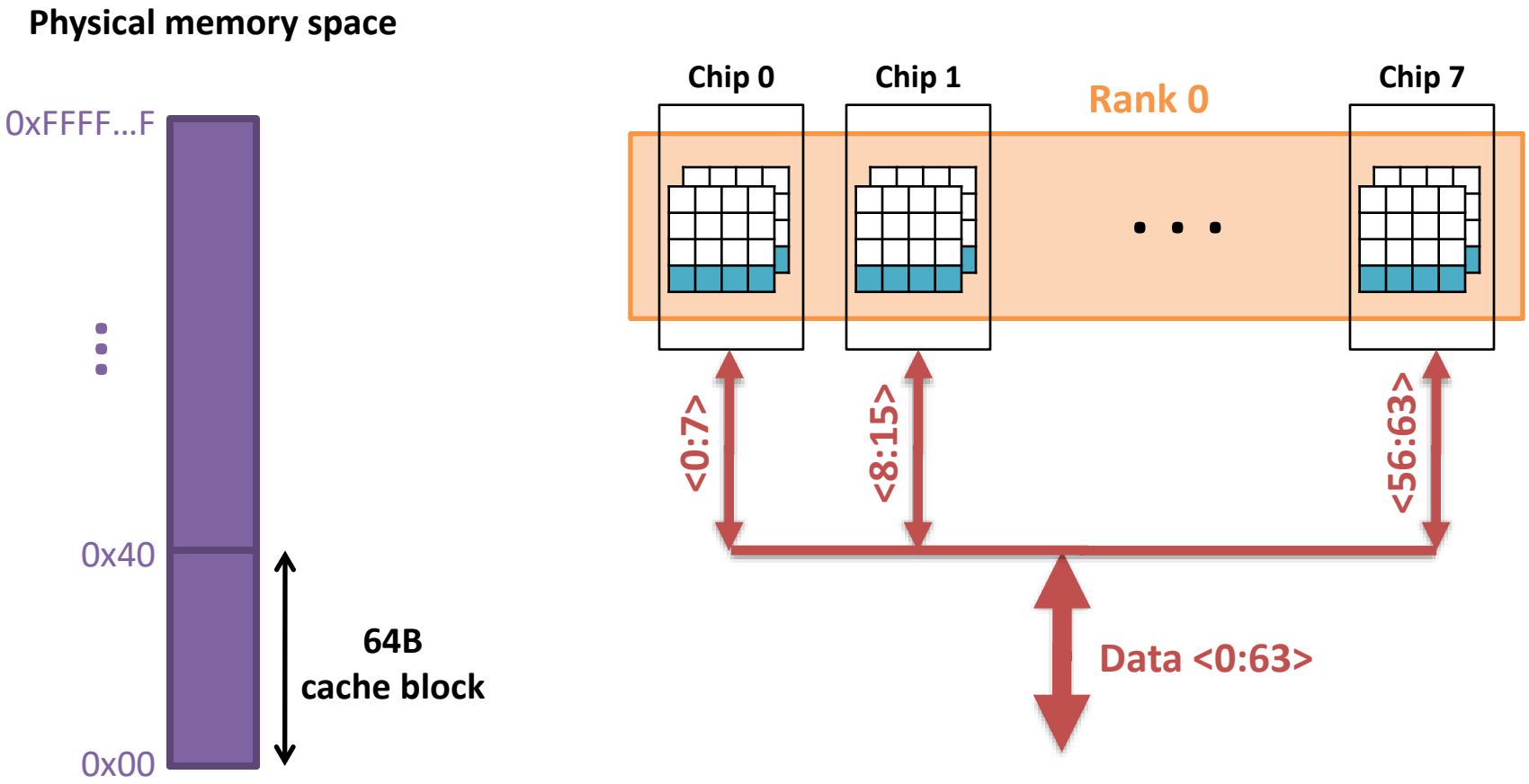
BANK的拆分



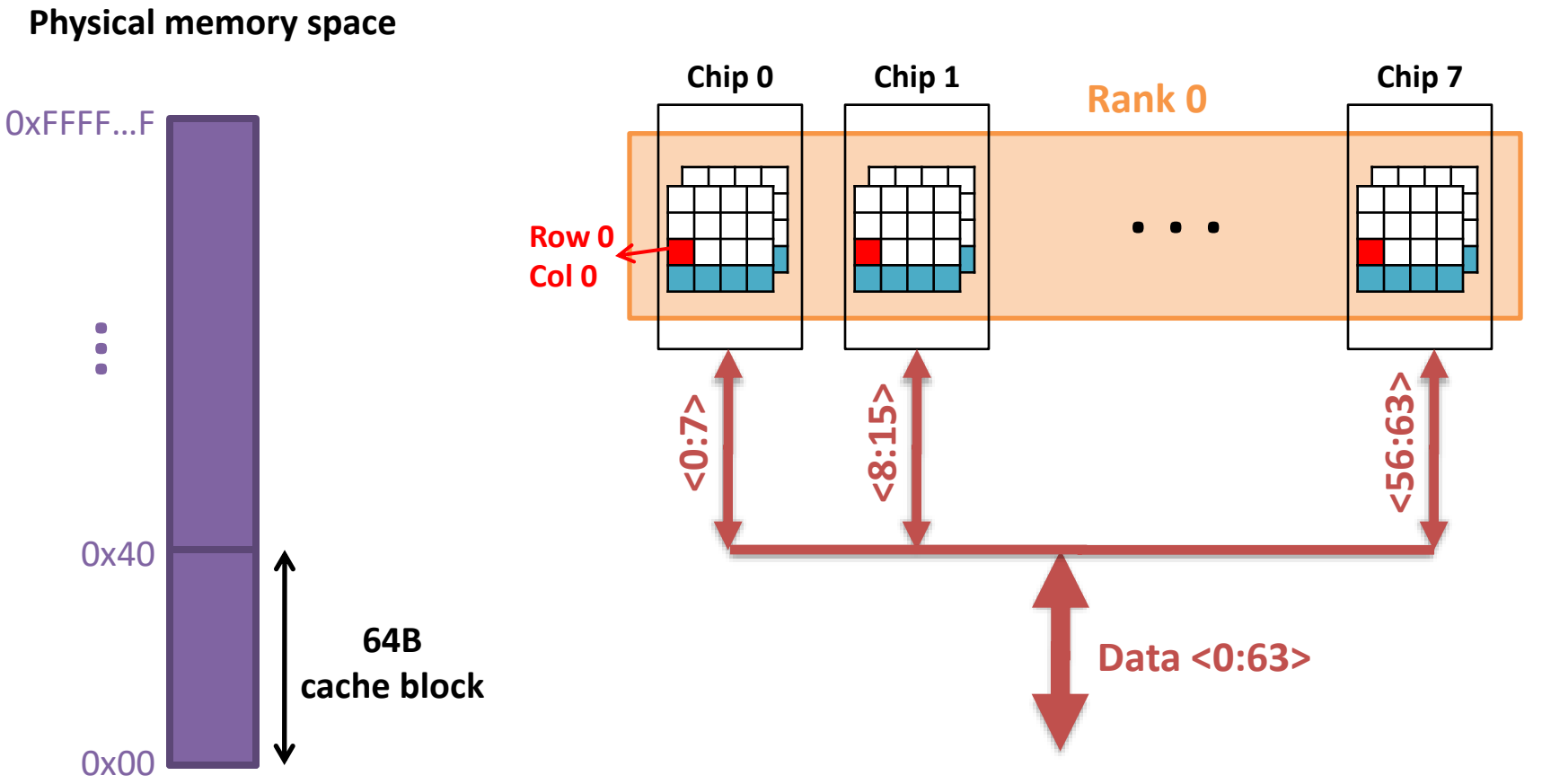
DRAM如何存储和读写数据?



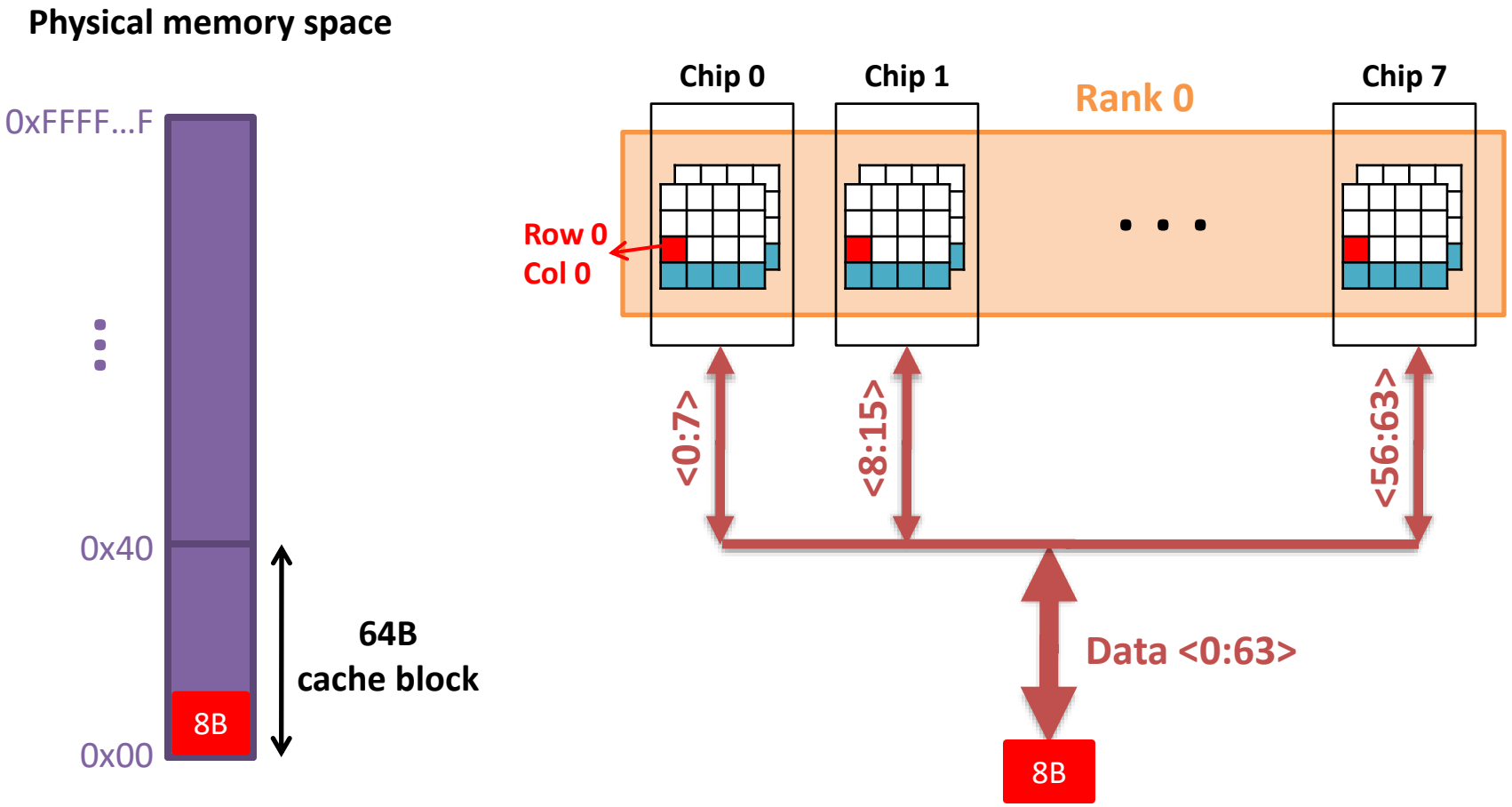
示例：传输缓存块


Physical memory space
0xFFFF…F
0x40
0x00
64B
cache block
Chip 0
Chip 1
Rank 0
Chip 7
<0:7>
<8:15>
<56:63>
Data <0:63>

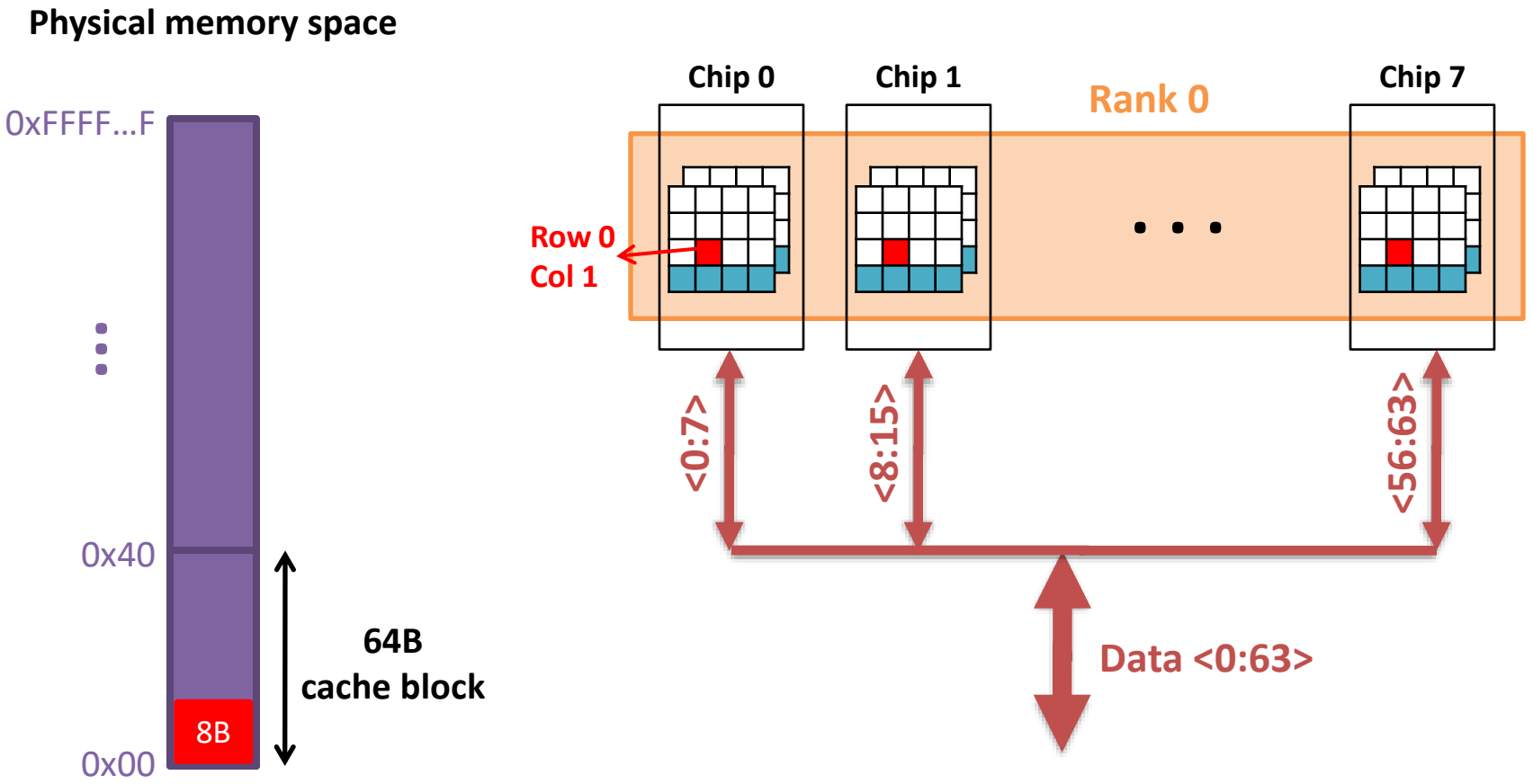
示例：传输缓存块


Physical memory space
0xFFFF…F
0x40
0x00
64B
cache block
Chip 0
Chip 1
Chip 7
Rank 0
Row 0
Col 0
<0:7>
<8:15>
<56:63>
Data <0:63>

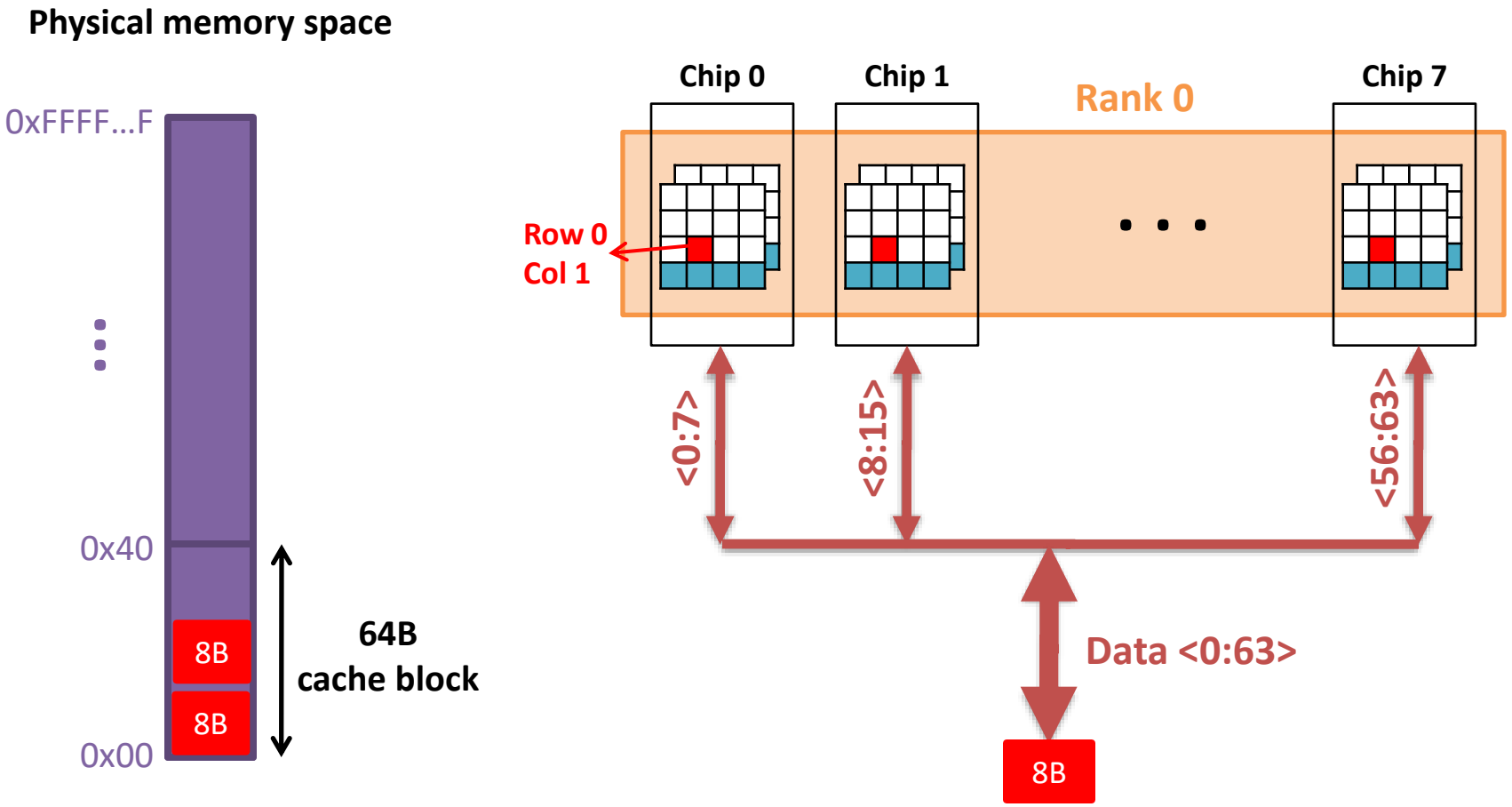
示例：传输缓存块


Physical memory space
0xFFFF…F
0x40
0x00
8B
64B
cache block
Chip 0
Chip 1
Chip 7
Rank 0
Row 0
Col 0
<0:7>
<8:15>
<56:63>
Data <0:63>
8B

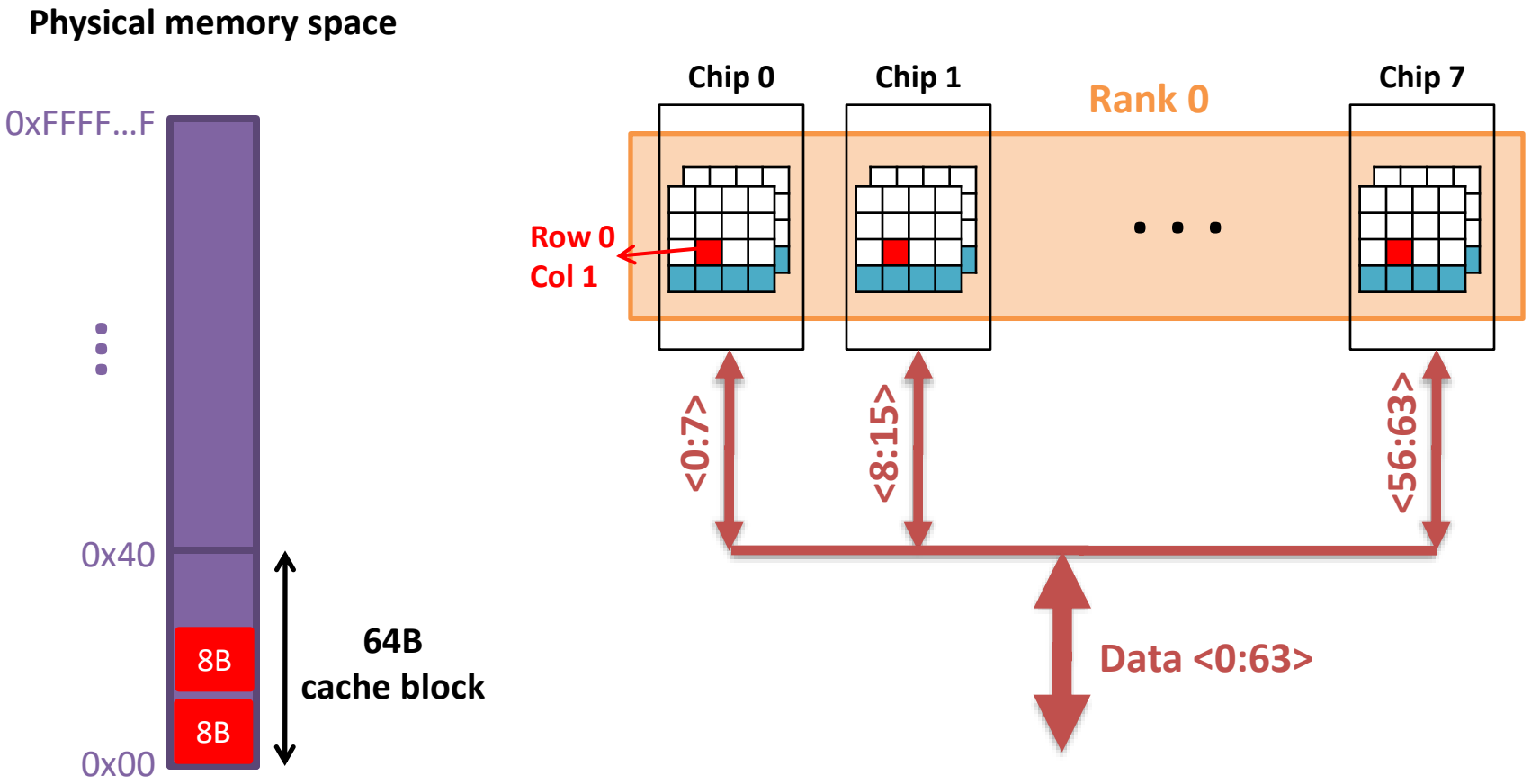
示例：传输缓存块


Physical memory space
0xFFFF…F
0x40
0x00
8B
64B
cache block
Chip 0
Chip 1
Rank 0
Chip 7
Row 0
Col 1
<0:7>
<8:15>
<56:63>
Data <0:63>

示例：传输缓存块


Physical memory space
0xFFFF…F
0x40
0x00
8B
8B
64B
cache block
Chip 0
Chip 1
Rank 0
Chip 7
Row 0
Col 1
<0:7>
<8:15>
<56:63>
Data <0:63>
8B

示例：传输缓存块



A 64B 缓存块需要8个 I/O 周期进行传输

在这个过程中, 8 列数据的读取是顺序进行的

Recap：存储层次架构的4个核心问题

Why Intel killed its Optane memory business

Effort to create a new tier of memory flopped as rivals offered faster and more open alternatives

Tobias Mann

Fri 29 Jul 2022 // 01:30 UTC

ANALYSIS Intel CEO Pat Gelsinger has confirmed that Intel will quit its Optane business, ending its attempt to create and promote a tier of memory that's a little slower than RAM but had the virtues of persistence and high IOPS.

The news should not, however, come as a surprise. The division has been on life support for some time following Micron's 2018 decision to terminate its joint venture with Intel, selling the fab in which the 3D XPoint chips that go into Optane drives and modules were made. While Intel has signaled it is open to using third-party foundries, without the means to make its own Optane silicon, the writing was on the wall.

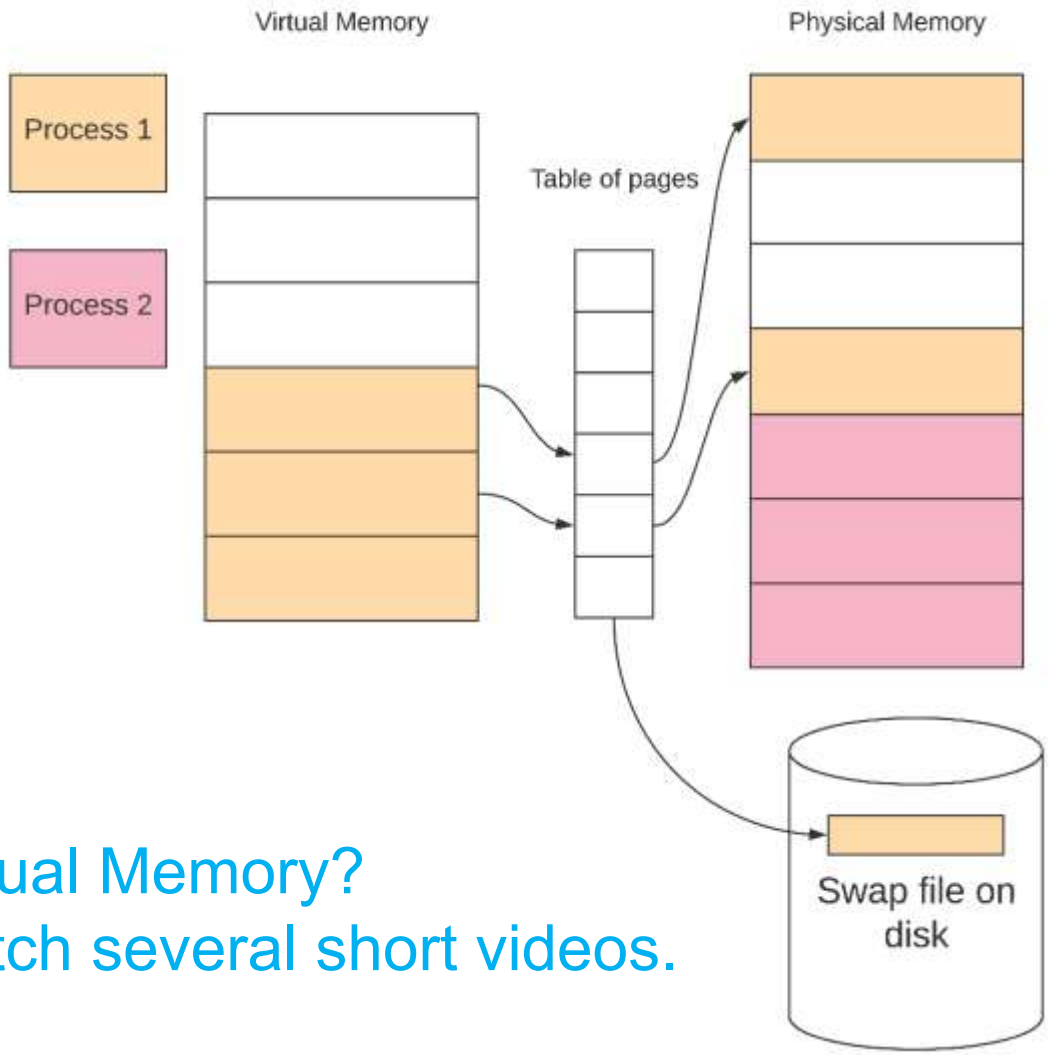
As our sister site Blocks and Files reported in May, the sale only came after Micron had saddled Intel with a glut of 3D XPoint memory modules – more than the chipmaker could sell. Estimates put Intel's inventories at roughly two years worth of supply.

In its poor earnings report for Q2, Intel said quitting Optane will result in a $559 million inventory impairment. In other words, the company is giving up on the project and writing off the inventory as a loss.

Placement

存储类型	Placement	备注
Registers	Anywhere; Int, FP, SPR	Compiler/programmer manages
Cache (SRAM)	Fixed in H/W	*Direct-mapped, Set-associative, Fully-associative*
DRAM	Anywhere	OS manages
Disk	Anywhere	OS manages

虚拟存储器



Why Virtual Memory?
Let's watch several short videos.

虚拟存储器

- 虚拟存储器的好处：
  - 主存成为整个存储层次架构中另外一层 (磁盘的缓存)
  - 使得程序能使用的地址空间或者工作集能够超过可用的物理内存的大小
    - 不再需要程序员来管理存储分配等繁琐的细节
    - 离散地使用大的地址空间成为可能
  - 允许多个用户程序**分时地共享**有限的内存空间和地址空间
- 虚拟存储器可以：
  - 提升昂贵内存的利用率
  - 提升程序开发效率 (对程序员友好)

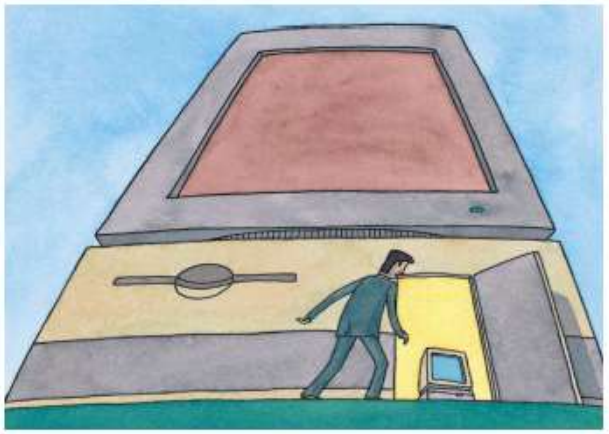
VM有无其他好处?

虚拟存储器

- 虚拟存储器的其它好处：
  - 每个程序认为当前内存里它是独占运行的 (假象)
    - 不需要管理多个程序使用地址空间的诸多问题
    - 例如，可以认为每个程序都从地址 0x0 开始
  - 内存保护易于实现
    - 每个程序都有自己私有的逻辑地址空间: 其它程序不可访问
  - 系统能获得更好的性能
    - 仅需从磁盘中加载一个程序当前所需的部分数据，就能够运行这个程序。

关于VM的更多细节

Virtual Memory: Issues of Implementation



The authors introduce basic virtual-memory technologies and then compare memory-management designs in three commercial microarchitectures. They show the diversity of virtual-memory support and, by implication, how this diversity can complicate and compromise system operations.

B. Jacob and T. Mudge, Virtual memory: issues of implementation, in *Computer*, 1998, vol. 31, no. 6, pp. 33-43.

(1) 后

- VM中，内存
  - *Page size*
  - Super pa
- 采用全相联
  - 操作系统负
  - 通过查询页
  - PT维持VA
    - VA：Vir
    - PA：Phy

Red Hat Training

A Red Hat training course is available for **Red Hat Enterprise Linux**

Chapter 14. Large Memory Optimization, Big Pages, and Huge Pages

Note

As explained in detail in this section, enabling big pages helps reduce TLB misses. However, this performance benefit is realized primarily when using large SGA sizes. Once a portion of memory is locked down for big pages, applications that use normal pages cannot access that portion of the memory. It is very important to make sure that there is enough memory for normal pages for applications and users to avoid excessive swapping. So, it is recommended that big pages be used only on systems that have large amounts of physical memory and for SGA sizes of 16GB or greater.

Big Pages in Red Hat Enterprise Linux 2.1 and Huge Pages in Red Hat Enterprise Linux 3, 4 and 5 are very useful for large Oracle SGA sizes and in general for systems with large amount of physical memory. It optimizes the use of Translation Lookaside Buffers (TLB), locks these larger pages in RAM, and the system has less bookkeeping work to do for that part of virtual memory due to larger page sizes. This is a useful feature that should be used on x86 and x86-64 platforms. The default page size in Linux for x86 is 4KB.

(1) 虚拟存储器的Placement

- 全相联是否意味着查找的代价巨大?
  - 对高速缓存(cache)来说，查找代价确实大；(硬件)
  - 因为，需要很多比较器来并行比较组内的Tag。
  - 此外，还需要宽度巨大的Multiplexer。
- 但是，对于VM来说，昂贵的查找可以通过页表（Page Table）来进行优化。
  - 页表就是一个 查找表 或 哈希表，消除并行查找开销。
  - 页表的查找引入了一层间接性 (导致了额外的访问延迟)
- 高代价的另外一个方面是延迟
  - 不过对于VM来说，延迟长一点不是问题。

(2) 虚拟存储器的Identification

Virtual PN	Physical PN	Dirty bit
0x20004000	0x2000	Y/N

- 类似于缓存中的tag store
  - 页表项主要包括：VA, PA, dirty bit, 等
- Virtual address:
  - 对共享系统中的多个程序来说，可以一样，且不会有冲突。
- Physical address:
  - 对程序员不可见，由OS进行管理；
  - 页面可以按照需求进行创建和删除，并且可以改变。
    - Demand Paging

(3) 虚拟存储器的Replacement

Second Chance (or Clock) Page Replacement Policy

Difficulty Level : Expert • Last Updated : 24 Aug, 2021

Prerequisite – Page Replacement Algorithms

Apart from LRU, OPT and FIFO page replacement policies, we also have the second chance/clock page replacement policy. In the Second Chance page replacement policy, the candidate pages for removal are considered in a round robin matter, and a page that has been accessed between consecutive considerations will not be replaced. The page replaced is the one that, when considered in a round robin matter, has not been accessed since its last consideration.

It can be implemented by adding a "second chance" bit to each memory frame-every time the frame is considered (due to a reference made to the page inside it), this bit is set to 1, which gives the page a second chance, as when we consider the candidate page for replacement, we replace the first one with this bit set to 0 (while zeroing out bits of the other pages we see in the process). Thus, a page with the "second chance" bit set to 1 is never replaced during the first consideration and will only be replaced if all the other pages deserve a second chance too!

(4) 虚拟存储器的Write Policy

- 通常采用write back策略
  - 因为，主存的下一级是Disk，而Disk的速度太慢，若采用write through会直接损害性能。
- 所以，页表为每个页面配置dirty bit
  - 硬件第一次写页面的时候需要对dirty bit置位
  - OS在进行替换的时候，需要检查dirty bit
  - Dirty bit为1的victim page需要写回下一级存储
    - 注意：磁盘写的延迟约为 10+ ms；
    - SSD的写延迟约为1.5ms (好的情况)

虚拟存储器的实现

- 缓存的实现采取固定的策略，用硬件FSM来进行控制。
- 虚拟存储器中页错误的开销与缓存缺失的开销差别很大
  - 磁盘访问延迟：10+ms
  - SSD最好也是1.5ms
  - 1.5ms约为3 Million 处理器周期@2GHz
- 因此，一些机制在设计的时候，工程选择就可以很不一样。
  - 虚拟存储器可以用软件来管理

(1) 页错误 - Page Faults

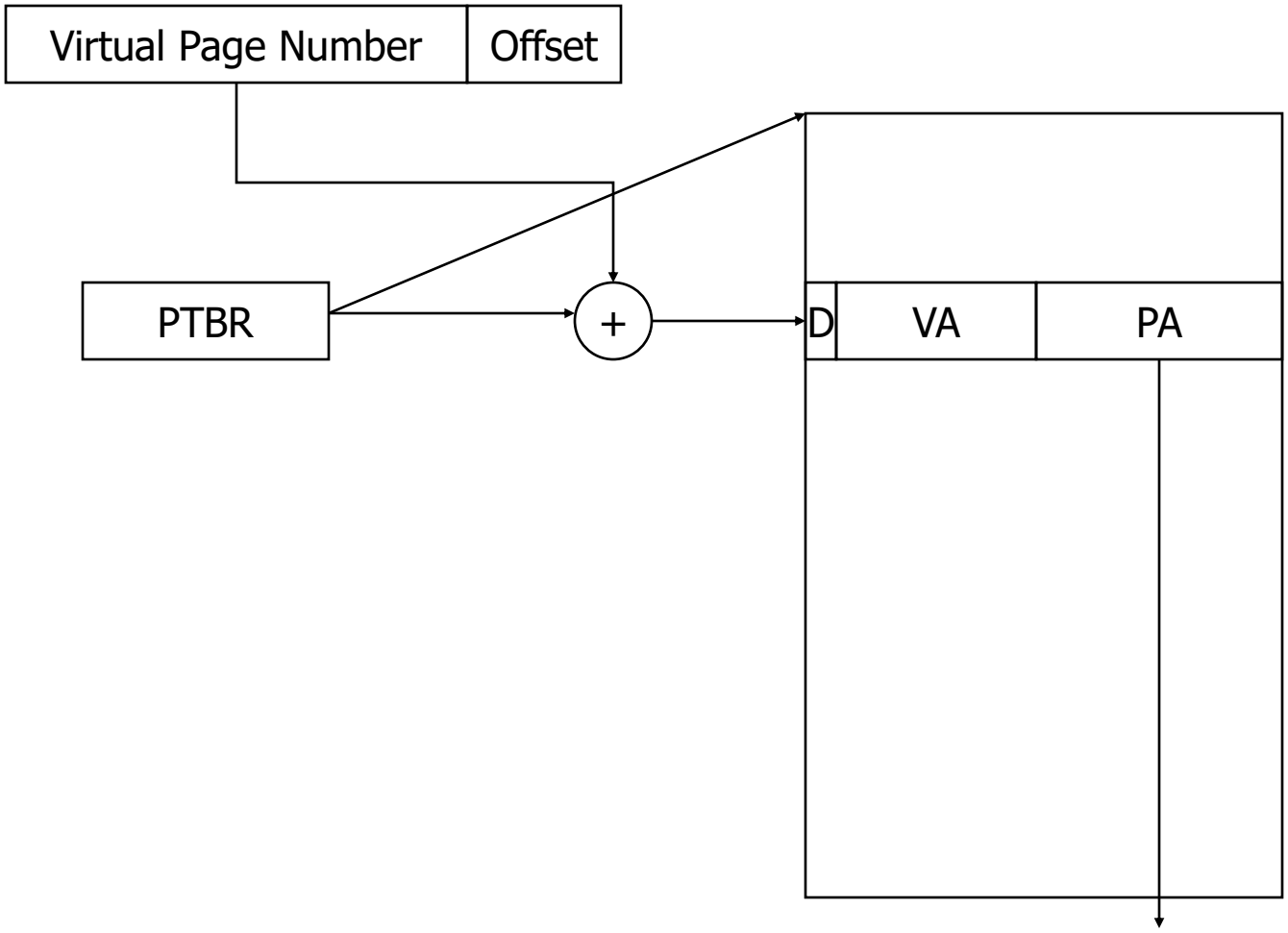
- 虚拟存储器中访问页面发生缺失称为：page fault，而不是Page Miss。
  - 要访问的页面 (page) 不在内存中
  - 这个时候就产生异常，需要保存PC等现场，进行上下文切换 (context switch)。
  - 调用OS中的page fault handler
    - 查找一个物理页面 (可能会发生替换)
    - 从磁盘中获取目标页面，导入内存。
  - 切换到另外一个处于就绪状态的任务去运行
  - 当磁盘访问完成时，发中断请求。
  - 中断获得响应后，重启原来的指令处理。

(2) 地址转换

VA	PA	Dirty	Ref	Protection
0x20004000	0x2000	Y/N	Y/N	Read/Write/Execute

- 操作系统和硬件通过PTE (Page Table Entry) 进行交互
- 那么，如何查找一个PTE?
  - &PTE = PTBR + page number * sizeof(PTE)
  - PTBR 对每个进程来说是私有的
    - PTBR的全称：Page Table Base Register
    - 每个进程都配置一个**逻辑**的PTBR
    - 上下文切换会替换 **物理**的PTBR 保存的内容

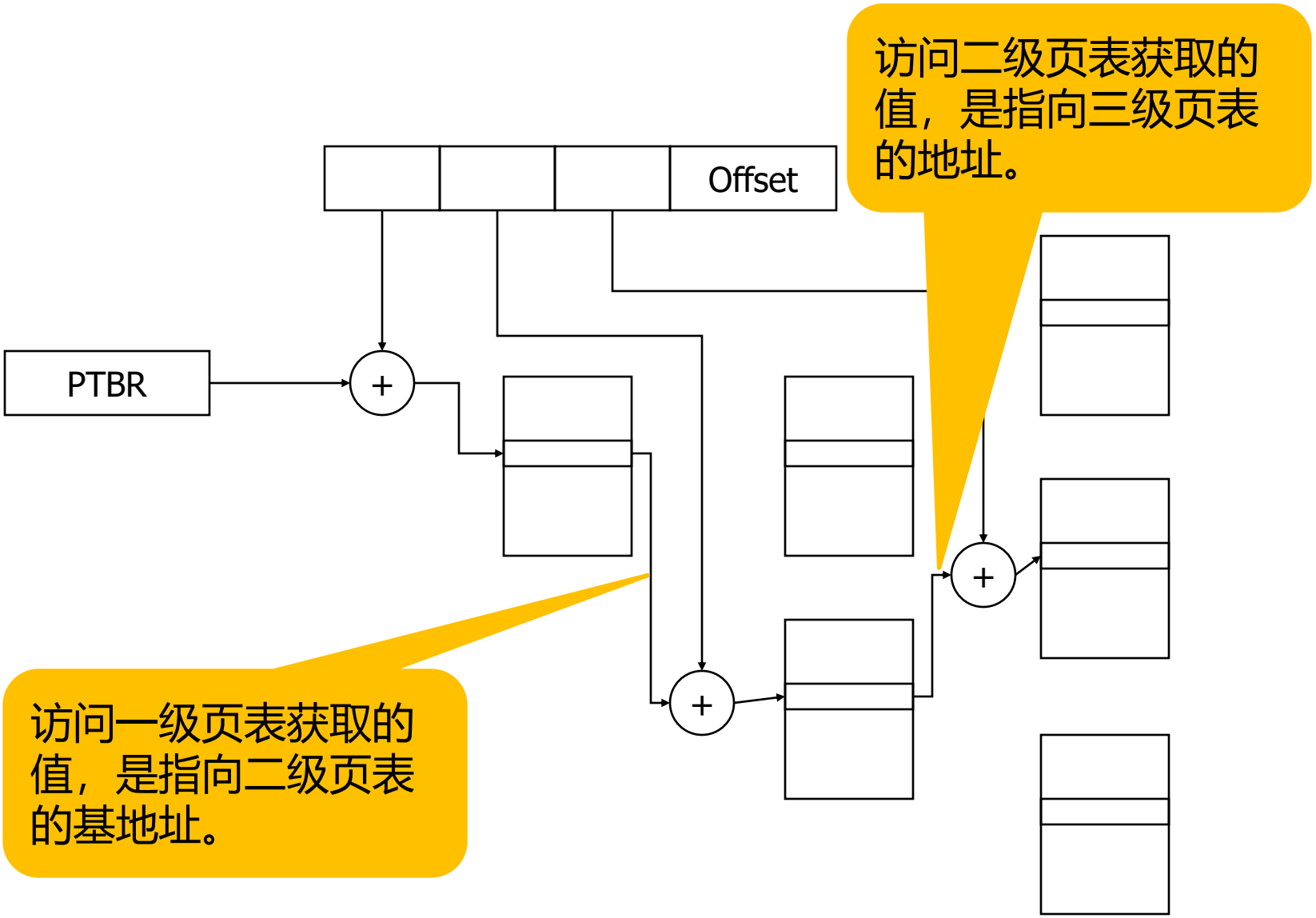
地址转换示意图



(3) 页表大小

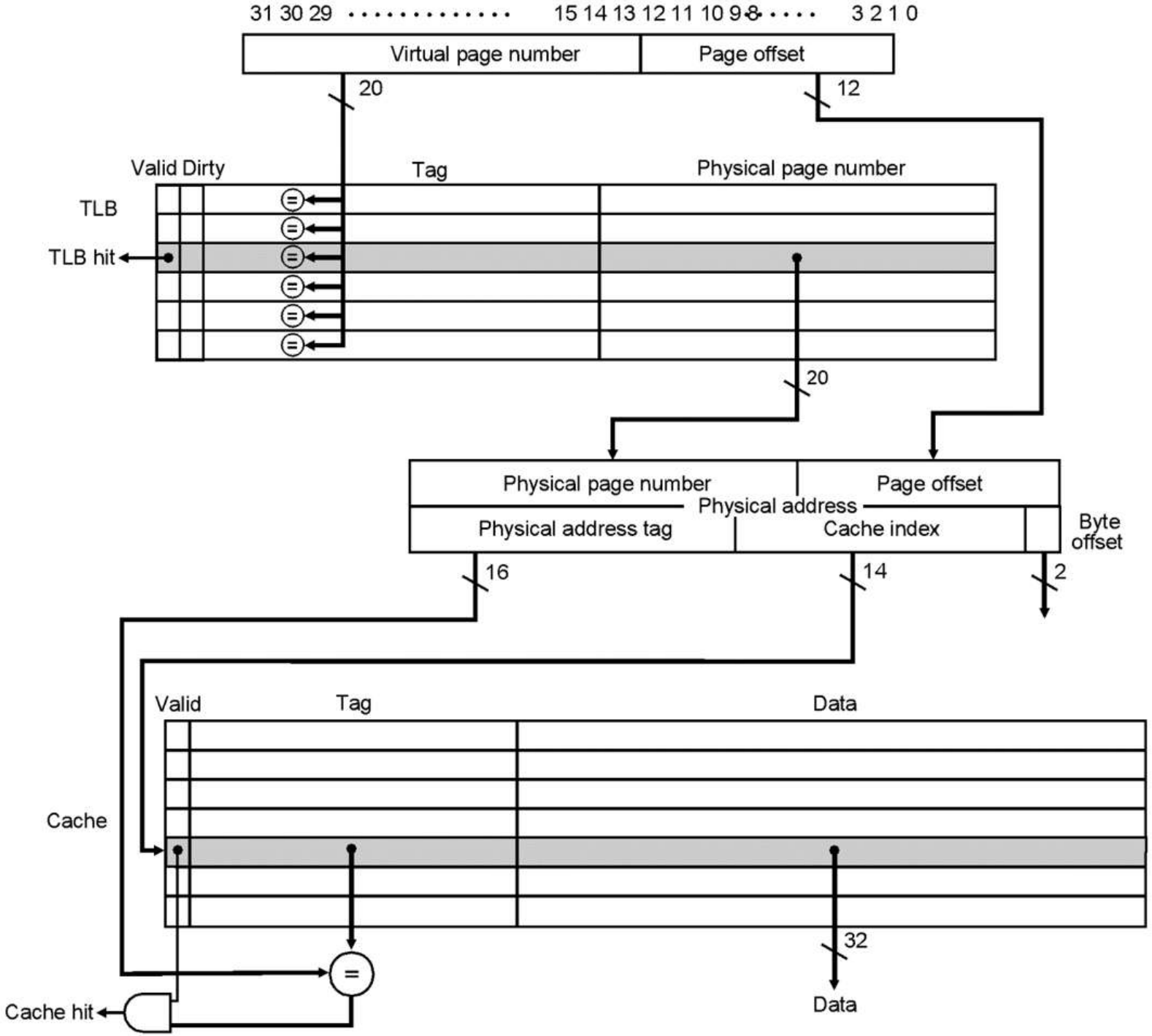
- 页表能有多大?
  - 2^{32} / 4K * 4B = 4M per program (!)
  - 对于64位机器来说，情况要糟糕很多。
- 如何减少页表的大小?
  - 使用多级页表
  - 使用inverted (hashed) page table
    - 本课程不做详细介绍
    - 感兴趣的同学可以自行调研

多级页表


访问二级页表获取的值，是指向三级页表的地址。
Offset
PTBR
+
+
+
访问一级页表获取的值，是指向二级页表的基地址。

(4) 利用快表加速地址转换

- 虚拟地址转换
  - PTE的访问会造成额外的内存访问
  - 每条指令(fetch/load/store)现在需要2次内存访问
    - 有可能更多, 对多级页表 或者 发生哈希冲突时
  - 即使对PTE进行缓存，依然很慢。
- 因此，当前CPU使用特殊的缓冲来保存PTEs
  - 这个结构叫：**TLB** (translation lookaside buffer)
  - 用于缓存当前一段时间内使用的PTEs
  - **期望：**可以利用页表访问的局部性来提升性能
  - **注意：**快表是用硬件实现的


31 30 29 ..... 15 14 13 12 11 10 9 8 ..... 3 2 1 0
Virtual page number
Page offset
20
12
Valid Dirty
Tag
Physical page number
TLB
TLB hit
=
20
Physical page number
Page offset
Physical address
Physical address tag
Cache index
Byte offset
16
14
2
Valid
Tag
Data
Cache
32
=
Cache hit
Data

(5) 虚拟存储器的保护

- 每个进程有自己私有的虚拟地址空间
  - 这样可自动地实现对恶意程序的防护

```
user@ubuntu:/opt/simplescalar_all/bin$ ls -l
total 14424
-rwxrwxrwx 1 user user   76538 May 27  2015 protoize
-rwxr-xr-x 1 root root 1000648 Jun 25  2018 sslittle-na-sstrix-addr2line
-rwxrwxrwx 2 root root  972952 Jun 25  2018 sslittle-na-sstrix-ar
-rwxrwxrwx 2 root root 1706868 Jun 25  2018 sslittle-na-sstrix-as
-rwxrwxrwx 1 user user   13644 May 27  2015 sslittle-na-sstrix-c++
-rwxr-xr-x 1 root root  132652 Jun 25  2018 sslittle-na-sstrix-c++filt
-rwxrwxrwx 1 user user   13644 May 27  2015 sslittle-na-sstrix-g++
-rwxr-xr-x 1 root root  154432 Jun 25  2018 sslittle-na-sstrix-gasp
-rwxrwxrwx 1 user user   67694 May 27  2015 sslittle-na-sstrix-gcc
-rwxrwxrwx 2 root root 1509844 Jun 25  2018 sslittle-na-sstrix-ld
-rwxrwxrwx 2 root root 1031028 Jun 25  2018 sslittle-na-sstrix-nm
-rwxr-xr-x 1 root root 1583964 Jun 25  2018 sslittle-na-sstrix-objcopy
-rwxr-xr-x 1 root root 1649248 Jun 25  2018 sslittle-na-sstrix-objdump
-rwxrwxrwx 2 root root  972948 Jun 25  2018 sslittle-na-sstrix-ranlib
-rwxr-xr-x 1 root root  330672 Jun 25  2018 sslittle-na-sstrix-readelf
-rwxr-xr-x 1 root root  907740 Jun 25  2018 sslittle-na-sstrix-size
-rwxr-xr-x 1 root root  891872 Jun 25  2018 sslittle-na-sstrix-strings
-rwxrwxrwx 2 root root 1583964 Jun 25  2018 sslittle-na-sstrix-strip
-rwxrwxrwx 1 user user   68716 May 27  2015 unprotoize
```

(6) 虚拟存储器的共享

- 通过下述方式共享内存位置:
  - 将共享的物理内存位置映射到共享者的虚拟地址空间里:
    - 例如，PA 0xC00DA对应的虚拟地址可以是:
      - VA 0x2D000DA for process 0
      - VA 0x4D000DA for process 1
  - Process 0 和 process 1均可以读写共享的位置：0xC00DA
- **请思考：**虚拟地址加共享会带来什么问题?
  - 培养计算机系统能力
  - 系统性思维（存储栈 与 计算栈）

关于虚拟缓存的有趣问题

- However, page sharing also causes homonym and synonym problem
  - homonym—同名异物
  - Synonym —异名同物



VA Homonyms – 同名异物

- Same VA to different PA (multitasking, multiprocessing environment)
- What is the problem?
  - Who is who? Who are you?
- How to solve the problem?
  - Flush cache on context switch
  - Add process ID to each tag
  - Use physically addressed caches

VA Synonyms – 异名同物

- Different VA to same PA (sharing)
  - Can end up with two copies of same physical line
- Solutions:
  - hardware synonym detection
  - flush cache on context switch
    - doesn't help for aliasing within address space
  - detect synonyms and ensure
    - all read-only, OR
    - only one synonym mapped
  - restrict VM mapping so synonyms map to same cache set